

QUERYBOT (DOCUMENT PROCESSING AI BOT)



Project ID - BSCS-F21-M-04

Project Advisor:

Dr. Muhammad Akram

Mujahid Program:

BS Computer Science

Shift:

Morning

Submitted By

Parihan Ayyaz

BSF2101300

Emaan Riaz

BSF2100969

Taimoor Hussain

BSF2106690

**QUERYBOT
(DOCUMENT PROCESSING AI BOT)**

BS in Computer Science 2022-2026

A project submitted in partial fulfillment of the requirements for
the award of the degree of **BS in Computer Science**

**Department of Information Sciences, Division of
Science and Technology, University of Education,
Township, Lahore**

April 30, 2026

© Copyright, 2026

Project Approval and Endorsement

“We hereby declare that we have read this project documentation and in our opinion this project is sufficient in terms of scope and quality for the award of the degree of BS in Computer Science.”

Project Primary Supervisor
Name: Mr. Suhail Ashfaq Butt
Designation: Assistant Professor
University of Education, Lahore

Project Examiner
Name:
Designation:

DECLARATION

We declare that this project title entitled “**QueryBot- Document Processing AI Bot**” is the result of our own research and development except as cited in the references. This project has not been accepted for any degree and is not concurrently submitted in candidate for any other degree. At any time if my statement is found to be incorrect even afterwards of BS in Computer Science the university has the right to withdraw my BS in Computer Science degree.

Signature: _____

Signature: _____

Name: HORIYA

Name: SEHAR SITARA

Date: April 30, 2026

Date: April 30, 2026

PLAGIARISM UNDERTAKEN

We solemnly declare that project work presented in this documentation entitles “**QueryBot- Document Processing AI Bot**” is solely our work with no significant contribution from any other person. Small contribution/help wherever taken has been acknowledged and that complete project has been written by us.

We understand that zero tolerance policy of the HEC and University of Education, Lahore towards plagiarism. Therefore, we as an author of the above titled project declare that no portion of our project documentation and any material used as reference is properly referred/ cited.

We undertake that if we are found guilty of any formal plagiarism in the above titled project even after award of BS degree, the University reserve the rights to withdraw/revoke our BS degree and that HEC and the University has the right to publish our names on the HEC/University Website on which names of students are placed who submitted plagiarized projects.

Signature: _____

Name: HORIYA

Date: April 30, 2026

Signature: _____

Name: SEHAR SITARA

Date: April 30, 2026

ACKNOWLEDGEMENT

We truly acknowledge the cooperation and help made by **Mr. Suhail Ashfaq Butt, Assistant Professor** from **University of Education**. He has been a constant source of guidance throughout the course of this project. We are also thankful to our friends and families whose silent support led us to complete our project.

- 1- Horiya
- 2- Sehar Sitara

Date: April 30, 2026

ABSTRACT

The escalating volume of digital documents across various sectors requires an efficient method for context-aware information retrieval. Traditional keyword-based search is often slow and fails to capture the complexity and context within large documents. This project introduces QUERYBOT, a web-based, AI-powered chatbot designed to resolve these limitations. QUERYBOT allows users to upload multiple document formats (PDF, Word, TXT) and interact with the content using natural language. The system utilizes advanced Natural Language Processing (NLP) techniques, including transformer-based models and semantic search, to convert documents into structured embeddings. This methodology enables the chatbot to provide accurate, real-time, and context-aware responses, even when dealing with complex data like tables. By ensuring accuracy, scalability, and multi-format support, QUERYBOT serves as a powerful solution for revolutionizing document analysis across education, business, healthcare, and legal fields.

List of Tables

2.1 Functional Requirements.....	10
2.2 Non-Functional Requirements.....	12
6.1 Register New User.....	59
6.2 Register Duplicate Email.....	60
6.3 Login with Valid Credentials.....	61
6.4 Login with Wrong Password.....	62
6.5 Upload Valid Document (PDF).....	63
6.6 Reject Valid File Type on Upload.....	64
6.7 Reject File Larger Than 10 MB.....	64
6.8 Search Documents in Library.....	65
6.9 Single Document Chat (Ask Question & Receive Reply).....	66
6.10 Generate Document Summary.....	67
6.11 Export Summary (PDF & DOCX).....	67
6.12 Multi-Document Chat (Select & Ask).....	68
6.13 Block Send in Multi-Chat When No Document Selected.....	68
6.14 Bookmark Chat Message & View in Bookmarks Page.....	69
6.15 Remove Bookmark from Bookmarks Page.....	69
6.16 Export Chat Transcript (PDF & DOCX).....	70
6.17 Delete Document (with Confirmation).....	70
6.18 Access Protected Page Without Login.....	71

List of Figures

3.1 User Use Case	22
4.1 Architecture Diagram.....	25
4.2 Entity Relationship Diagram.....	28
4.3 Data Flow Diagram Level 0.....	30
4.4 Data Flow Diagram Level 1.....	32
4.5 Class Diagram	35
4.6 User Registration & Login Sequence.....	36
4.7 Document Upload & Processing Sequence	37
4.8 Query Processing Sequence (RAG + LLM).....	38
4.9 Document Summary Generation Sequence.....	39
5.1 Login Screen.....	41
5.2 Registration Screen.....	42
5.3 Home Page.....	43
5.4 Documents Dashboard & Upload Screen.....	44
5.5 Document Upload Progress State.....	44
5.6 Delete Document Action.....	45
5.7 Single-Document Chat Screen.....	45
5.8 Edit Message & Regenerate Response.....	46
5.9 Chat History, Rename & Delete Session	47
5.10 Document Search Functionality.....	47
5.11 AI Response Thinking State.....	48
5.12 Live Document Preview.....	48
5.13 Generate Summary Feature.....	49
5.14 Summary & Export Controls.....	49
5.15 Multi-Document Chat & Multi-File Preview.....	50
5.16 Bookmarks Management.....	50
5.17 User Profile & Logout Controls.....	51
5.18 Citations & References View.....	51
5.19 Login Form Inline Validation (Email & Password).....	53
5.20 Document Upload Validation Invalid Type & Oversized File.....	54

TABLE OF CONTENTS

Title.....	i
Statement of Submission.....	iv
Declaration.....	v
Plagiarism Undertaken.....	vi
Acknowledgement.....	vii
Abstract.....	viii
Table of Content.....	xi
List of Tables.....	ix
List of Figures.....	x
CHAPTER 01: GATHERING AND ANALYZING INFORMATION.....	1
Introduction.....	2
Problem Statement.....	2
Project Goals & Objectives.....	2
Research Questions.....	3
Methodology.....	3
Definitions, Acronyms and Abbreviations.....	5
CHAPTER 02: SOFTWARE REQUIREMENT SPECIFICATION.....	7
Stakeholders Characteristics.....	8
Domain Requirements.....	8
Functional Requirements.....	10
Non-Functional Requirements.....	12
CHAPTER 03: ANALYSIS.....	15
Introduction.....	16
User Use Cases.....	16
User Use Case Diagram.....	22
CHAPTER 04: DESIGN.....	23
Architecture Diagram.....	24
Entity Relationship Diagram.....	26
Data Flow Diagrams.....	29
Class Diagram.....	33
Sequence Diagram.....	36
CHAPTER 05: GRAPHICAL USER INTERFACES.....	40

Introduction.....	41
Overall Interface Structure.....	41
Core Interface Screens.....	41
Login Screen.....	41
Registration Screen.....	42
Home page.....	42
Document Dashboard & Upload Screen.....	43
Document Upload Progress State.....	44
Delete Document Action.....	45
Single Document Chat Screen.....	45
Edit Message & Regenerate Response.....	46
Chat History,Rename & Delete Session.....	46
Search Functionality.....	47
AI Response Thinking State.....	47
Live Document Preview.....	48
Gernerate Summary Feature.....	48
Summary & Export Control.....	49
Multi-Document Chat & Multi-File Preview.....	49
Bookmarks Management.....	50
User Profile & Logout Controls.....	50
Citations & Reference View.....	51
Navigation & Workflow.....	51
Form Validation & Error Message.....	52
Login Form Inline Validation.....	52
Document Upload Validation (Type & Size).....	53
CHAPTER 06 : TESTING.....	55
Introduction.....	56
Testing strategy.....	56
Types of Tesing Performed.....	56
Testing Plan.....	57
Test Cases.....	59
Test Case Number: 1.....	59
Test Case Number: 2.....	60
Test Case Number: 3.....	61

Test case Number: 4.....	62
Test case Number: 5.....	63
Test case Number: 6.....	64
Test case Number: 7.....	64
Test case Number: 8.....	65
Test case Number: 9.....	66
Test case Number: 10.....	67
Test case Number: 11.....	67
Test case Number: 12.....	68
Test case Number: 13.....	68
Test case Number: 14.....	69
Test case Number: 15.....	69
Test case Number: 16.....	70
Test case Number: 17.....	70
Test case Number: 18.....	71
Error Handling & Debugging.....	72
Limitations Found During Testing.....	73
CHAPTER 07: CONCLUSION & FUTURE WORK.....	75
Introduction.....	76
Summary of Fundings & Achievements.....	76
Challenges & Limitations.....	76
Future Work & Recommendations.....	78
Conclusion.....	79

CHAPTER 01

Gathering & Analyzing Information

1.1 Introduction

QueryBot is an AI-powered web platform designed to simplify how users interact with digital documents. In today's digital world, individuals, students, and organizations handle a large number of files such as PDFs, Word documents, reports, and manuals. Manually reading long documents to find specific information is time-consuming and inefficient. QueryBot aims to solve this problem by allowing users to upload documents and ask questions in natural language. The AI system understands the content and responds with accurate, context-based answers. This platform will serve as a single solution for document search, automated summaries, and intelligent content extraction for students, teachers, businesses, and professionals.

1.2 Problem Statement

With the rapid digitalization in the World and growing reliance on online learning, businesses, and documentation, people struggle to extract information quickly from large documents. Many users face the following challenges:

- Lack of a reliable platform for AI-based document searching
- Difficulty in reading long research papers, reports, or contracts
- Time wasted locating specific information inside documents
- No local platform that supports multi-format document processing
- Students and professionals often need summaries or explanations but do not have AI assistance

There is a need for an online AI system in Pakistan that can intelligently process documents, answer user queries, generate summaries, and improve accessibility to information.

1.3 Project Goals & Objectives

The primary goal is to develop an AI-based web system that processes uploaded documents and provides fast, accurate, and human-like responses by understanding document content.

1.3.1 Objectives

- a. To provide an online platform for AI-powered document processing.
- b. To allow users to upload multiple file formats (PDF, DOCX, TXT).
- c. To integrate AI for generating summaries and answering questions.
- d. To support multi-document analysis and document comparison.

- e. To offer a user-friendly chat interface for document-based conversations.
- f. To enable students and professionals to easily extract relevant information.
- g. To implement semantic search for accurate and context-aware results.
- h. To reduce the manual effort required to read lengthy documents.
- i. To support educational, business, healthcare, and legal domains.

1.4 Research Questions

1. What factors do users consider important when interacting with an AI document assistant?
2. What features are essential in an AI-based document processing platform?
3. What unique value can QueryBot provide that existing tools lack?
4. How big is the global and local demand for AI-powered document assistants?
5. What percentage of students and professionals prefer AI tools for document analysis?
6. What technical features can improve the accuracy of AI responses?
7. What functionalities do corporate or educational users expect from QueryBot?
8. How can a smart document assistant improve learning and productivity through summaries and insights?

1.5 Methodology

1.5.1 Chosen Methodology

For this project, we have selected the Agile Software Development Model. Agile is an iterative and flexible approach that divides the project into small manageable cycles called sprints. Each sprint involves planning, designing, developing, testing, and reviewing a small portion of the system. **Agile** is most suitable for QueryBot because the project involves AI-based features that may require adjustments based on testing accuracy, user feedback, and model improvements. The flexibility of Agile allows continuous refinement of the chatbot responses, document processing pipeline, and user interface. **Agile** ensures smooth collaboration, early testing, continuous improvement, and rapid adaptation to new requirements. Each iteration will help us improve accuracy, fix issues quickly, and deliver a more reliable and intelligent document processing system.

1.5.2 Available Methodology

Other available methodologies rather than the waterfall model include:

a. Waterfall Model

Waterfall is a linear model in which each phase must be completed before moving to the next. It is suitable for projects with stable requirements. However, for QueryBot, we expect iterative refinement, especially in AI features, so Waterfall is not ideal.

b. Scrum Framework

Scrum is a subset of Agile and works in short sprints with defined roles like Product Owner and Scrum Master. Although highly effective, it requires specialized roles and continuous stakeholder involvement. Since our project team is small, pure Scrum may add unnecessary complexity.

c. Incremental Development Model

This model builds software in small increments. However, unlike Agile, increments are delivered in fixed sequences and do not support frequent requirement changes. QueryBot requires improvements based on testing and feedback, so Incremental is not the best choice.

1.5.3 Reasons for Chosen Methodology

The following are the reasons leading to the chosen methodology:

a. Flexibility and Adaptability:

Agile allows us to modify or improve AI responses, embeddings, and document analysis techniques during development.

b. Iterative Improvement:

Each sprint helps test the system early, identify issues, and refine features like semantic search, chatbot UI, and document processing.

c. Faster Testing and Feedback:

We can test small modules quickly, such as file upload, embeddings generation, or chatbot responses, and improve them without waiting for the whole system to finish.

d. Better Risk Management:

AI projects often involve uncertainty in output quality. Agile enables us to handle risks early by experimenting in short cycles.

e. User-Centric Development:

Agile focuses on user needs. We can gather user expectations and adjust QueryBot's accuracy, speed, and interface accordingly.

f. Continuous Integration:

Agile supports gradual integration of AI libraries, vector databases, and NLP models without disrupting the entire project. Definitions, Acronyms and Abbreviations.

1.6 Definitions, Acronyms and Abbreviations

1.6.1 Definitions

i. Document Processing

The automated method of reading, analyzing, and extracting information from digital documents such as PDFs, Word files, and text files.

ii. Embeddings

A numerical representation of text used by AI models to understand meaning, context, and similarity within documents.

iii. Semantic Search

A search technique that understands the meaning behind words, allowing the system to find contextually relevant answers instead of simple keyword matches.

iv. Natural Language Processing (NLP)

A field of Artificial Intelligence that enables computers to understand, interpret, and generate human language in a meaningful way.

v. AI Chatbot

An intelligent system that interacts with users through text, answering questions and providing information based on document content.

vi. Vector Database

A specialized database used to store and retrieve embeddings for fast and accurate similarity-based searching.

vii. Document Ingestion

The process of uploading, reading, and converting documents into machine-understandable formats for further processing.

viii. Retrieval-Augmented Generation (RAG)

A technique where AI retrieves relevant information from documents and combines it with language generation to produce accurate, context-based answers.

1.6.2 Acronyms and Abbreviations

- i.** AI: Artificial Intelligence
- ii.** NLP: Natural Language Processing
- iii.** Q/A: Question and Answering
- iv.** PDF: Portable Document Format
- v.** DOCX: Microsoft Word File Format
- vi.** RAG: Retrieval-Augmented Generation
- vii.** API: Application Programming Interface
- viii.** DB: Database
- ix.** UI: User Interface
- x.** UX: User Experience
- xi.** JSON: JavaScript Object Notation
- xii.** LMS: Language Model System (AI Model)
- xiii.** ETL: Extract, Transform, Load (Document Processing Flow)

CHAPTER 02

Software Requirement Specification

2.1 Stakeholders Characteristics

For QueryBot, the stakeholders include all individuals and groups who interact with, benefit from, or influence the AI-based document processing system.

1. Primary Users (Students, Professionals, Researchers)

These users rely on QueryBot to extract information from large documents quickly. Their needs include fast document uploads, accurate question answering, automated summaries, and context-aware insights. Their feedback plays a major role in improving system accuracy, response quality, and overall usability. Training, FAQ support, and user guidance will ensure smooth adoption across varying technical skill levels.

2. Organizations & Institutions (Secondary Stakeholders)

Universities, companies, and research teams benefit from QueryBot's ability to process reports, policies, manuals, and research material. They require secure document handling, collaborative features, and reliable performance for large files. Their evaluation is based on system accuracy, integration capability, and improvements in productivity.

3. Project Team

The project team includes AI/ML engineers, backend developers, frontend developers, database specialists, and UI/UX designers. Their tasks involve designing models, integrating vector databases, developing embeddings pipelines, and ensuring that QueryBot answers accurately and efficiently. Success requires coordination, resource availability, proper tools, and strong communication among team members to handle iterative improvements and scalability needs.

2.2 DOMAIN REQUIREMENTS

Domain requirements describe the essential functionalities and constraints for an AI-based document processing system.

1. Document Interaction Requirements

The solution must understand user questions and retrieve content from documents, including PDF, DOCX, and text files. It should provide precise, context-aware answers based on semantics rather than keywords. The system must handle complex queries regarding definitions, summaries, tables, and structured content.

2. AI & NLP Processing Requirements

QueryBot must utilize advanced NLP techniques for embeddings, semantic search, summarization, and intent detection. It should maintain accuracy across different document types, support domain-specific vocabulary, and adapt as more documents are uploaded.

3. User Experience Requirements

The system must provide a clean, intuitive interface where users can upload files, ask questions, and view results effortlessly. The UI should support multi-document chat, highlight relevant text passages, and allow easy navigation.

4. Scalability & Performance Requirements

QueryBot should support large document uploads, multiple concurrent users, and fast response times under 1 second. The platform must scale horizontally with increasing file sizes and user load, ensuring stable performance.

2.3 Functional Requirements

Table 2.1: Functional Requirements

No	Requirement	Description
FR1	Document Upload System	➤ Allows users to upload files in multiple formats (PDF, DOCX, TXT) and prepares them for processing.
FR2	AI-Powered Semantic Search	➤ Uses embeddings to understand meaning and retrieve accurate, context- based answers from documents.
FR3	Natural Language Processing (NLP)	➤ Enables QueryBot to understand user questions, detect intent, summarize content, and generate responses.
FR4	Multi-Document Handling	➤ Users can upload multiple documents and QueryBot retrieves answers from all relevant files.
FR5	Summarization Engine	➤ Provides short, detailed, or section-based summaries of uploaded documents.
FR6	User Authentication	➤ Ensures secure login and personalized document sessions for each user.

2.3.1 Description of Functional Requirements

1. Document Upload System

This module allows uploading PDF, Word, and text files. It handles file preprocessing, text extraction, cleaning, and metadata generation. The system uses secure storage and ensures documents are processed efficiently for embeddings generation.

2. AI-Powered Semantic Search

Semantic search retrieves relevant passages from documents by understanding context rather than matching exact keywords. In the current implementation, semantic retrieval configuration (embedding model and retrieval parameters) is managed at the system configuration level; dashboard-based retraining and quality control are planned for future versions.

3. Natural Language Processing (NLP)

The NLP pipeline interprets user queries and generates meaningful responses using retrieved document context. QueryBot supports question answering and context-grounded response generation, while advanced intent-classification and feedback-driven continuous learning may be added in future versions.

4. Multi-Document Handling

Users can interact with several documents simultaneously. The system performs retrieval across selected files, merges relevant context from multiple sources, and returns context-aware answers without forcing the user to query each file separately.

5. Summarization Engine

The system generates summaries from uploaded document content through an AI-based summarization flow. The current implementation focuses on context-grounded summaries, while advanced customization options (e.g., style/tone/section-specific formats) can be considered for future enhancement.

6. User Authentication

The system supports secure registration and login using email and password with hashed credential storage. JWT-based authorization protects backend APIs and ensures each user accesses only their own documents, sessions, and generated outputs.

2.4 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

No	Requirement	Description
NFR1	Security	➤ Ensures protection of uploaded documents, user data, and API communication using encryption and secure access controls.
NFR2	Usability	➤ Must provide a clean, simple, and intuitive interface for efficient interaction.
NFR3	Logging & Monitoring	➤ Tracks system performance, errors, document processing status, and user interactions for debugging and growth.
NFR4	Accessibility	➤ Supports all users, including those with challenges, through readable text, clean UI, and optional speech output.
NFR5	Scalability	➤ Handles increasing documents, users, and embeddings without performance issues.
NFR6	Performance	➤ Ensures fast response time (under 1 second) and optimized AI output even under load.
NFR7	Maintainability	➤ Allows easy updates, debugging, and model retraining with modular architecture.
NFR8	Reliability	➤ Maintains continuous availability, uptime, and consistent AI accuracy even during heavy loads.

2.4.1 Description of Non-Functional Requirement

1. Security

Includes secure authentication, protected API access, hashed password storage, and controlled access to user-specific documents and chat sessions. API communication should be deployed over secure transport in production environments. The system applies practical safeguards for MVP-level privacy and access protection.

2. Usability

Provides a clean interface, straightforward navigation, and guided workflows for uploading documents, asking questions, reading citations, and viewing summaries. The frontend layout is designed to minimize user confusion and support smooth interaction.

3. Logging and Monitoring

Tracks API/runtime errors, document processing failures, and retrieval/response behavior through application logging. These logs support debugging, issue diagnosis, and gradual quality improvement of the platform.

4. Accessibility

Supports readability through clear typography, consistent layout, and understandable interaction flow. The current implementation emphasizes practical usability; advanced accessibility features (e.g., speech output or full WCAG optimization) can be added in future versions.

5. Scalability

Uses vector indexing and retrieval architecture suitable for increasing document collections and user queries. The design is extensible for future infrastructure scaling, although distributed load-balancing architecture is outside the current MVP scope.

6. Performance

Targets an interactive response experience for document retrieval and question answering under typical usage conditions. Actual latency depends on document size, retrieval depth, and external LLM API response time; therefore, strict sub-second guarantees are not assumed.

7. Maintainability

The system follows a modular backend/frontend structure that supports easier feature enhancement, bug fixing, and configuration updates. This architecture allows future improvements such as model replacement, retrieval tuning, and interface refinement with limited disruption.

8. Reliability

Provides dependable operation through consistent route behavior, controlled exception handling, and persistent storage of documents, vectors, and chat sessions. The platform is designed for stable usage in its current scope, with high-availability/failover capabilities considered future enhancements.

CHAPTER 03

Analysis

3.1 Introduction

The analysis phase translates project requirements into clear system behavior by identifying user goals, system boundaries, interaction flows, and processing logic. For QueryBot, analysis focuses on how a registered user interacts with a document-centric AI assistant to upload files, generate embeddings, ask context-aware questions, receive source-grounded responses, and maintain persistent chat sessions.

QueryBot is implemented as a web-based client-server system where the frontend (Next.js) manages user interaction and the backend (FastAPI) handles authentication, document processing, retrieval-augmented generation (RAG), citation mapping, and chat/session persistence. Since the system does not include an administrative role, all analysis artifacts in this chapter model only the User as the primary actor and the System as the supporting actor that validates requests, executes processing pipelines, and returns results.

3.2 User Use Cases

UC Number: UC_QB_01

UC Name: Register Account

Primary Actor: User

Secondary Actor: System (Authentication Service, Database)

Description:

The user creates a new account with email and password so they can access protected QueryBot features.

Preconditions:

- User can open the application.
- User is not already logged in (or chooses to register).

Main Success Scenario:

- User opens the Register page.
- User enters email and password.
- User accepts Terms of Service / Privacy Policy (required on register screen).
- System validates email format and password rules (minimum length and special character) via client-side checks.
- User submits the form.
- System sends POST /auth/register with email and password.
- System checks that the email is not already registered.
- System stores the user with hashed password in the database.
- System automatically performs login (POST /auth/login) and receives a JWT access token.
- System stores the token for authenticated requests.
- User is redirected to /documents.

Alternative Scenario:

- Email already exists → API returns error; UI shows “Email already registered” (or equivalent server message).
- Invalid email / weak password → UI shows inline validation before submit.
- Terms not accepted → UI shows validation message; registration does not proceed.
- Backend unavailable or request fails → UI shows error message; user may retry.

Postcondition:

- A new user record exists and the user is authenticated with a valid JWT, ready to use documents and chat.

UC Number: UC_QB_02

UC Name: Login

Primary Actor: User

Secondary Actor: System (Authentication Service, Database)

Description:

The user signs in with an existing email and password to obtain a JWT and access protected areas of QueryBot.

Preconditions:

- User can open the application.
- User has a registered account (same email/password pair stored in the system).

Main Success Scenario:

- User opens the Login page.
- User enters email and password.
- System validates email format and password rules via client-side checks (before calling the API).
- User submits the form.
- System sends POST /auth/login with email and password.
- System verifies credentials against the hashed password in the database.
- System returns a JWT access token.
- System stores the token for authenticated requests.
- User is redirected to the intended route (default /documents, or next query if used).

Alternative Scenario:

- Wrong email or password → API returns 401; UI shows “Incorrect email or password.”
- Invalid email / password not meeting rules → inline validation; login request is not sent until fixed.
- Backend unavailable → UI shows error; user may retry.

Postcondition:

- User holds a valid JWT and is authorized to access documents, chat, and related protected features.

UC Number: UC_QB_03

UC Name: Upload Document (PDF/DOCX/TXT)

Primary Actor: User

Secondary Actor: System (Document Service, Database, Parser Service)

Description: User uploads a supported document, and the system stores it, extracts text, and prepares it for AI retrieval.

Preconditions:

- User is authenticated.
- File is in supported format and within size limit.

Main Success Scenario:

- User selects a file from local device.
- System validates extension and file size.
- System saves file to storage.
- System extracts text content from the file.
- System creates document metadata record.
- System triggers indexing preparation for retrieval.
- System confirms successful upload and updates document list.

Alternative Scenario:

- Unsupported file type -> system rejects upload with validation message.
- File too large -> system rejects upload and shows limit warning.
- Extraction failure/corrupted file -> system returns readable processing error.

Postcondition:

Document is available in user's library for preview, querying, and summarization.

UC Number: UC_QB_04

UC Name: View Document Preview

Primary Actor: User

Secondary Actor: System (Preview Service, File Storage)

Description: User opens document preview to inspect uploaded content before or during querying

Preconditions:

- User is authenticated.

Selected document belongs to user and exists in storage

Main Success Scenario:

- User clicks preview for a document.
- System verifies user ownership.
- System checks file type.
- For PDF, system streams file directly.
- For DOCX, system serves converted preview PDF if available.
- Preview is shown in the interface.

Alternative Scenario:

- Preview file missing -> system returns "Preview not found."
- Original file missing -> system returns "File not found on server."
- Unauthorized access attempt -> system denies request.

Postcondition:

User can view document content in preview mode.

UC Number: UC_QB_05**UC Name: Initiate Semantic Search**

Primary Actor: User

Secondary Actor: System (RAG Service, Vector Store, LLM Service, Database)

Description: User asks a natural-language question, and QueryBot returns a context-grounded answer from uploaded documents.

Preconditions:

- User is authenticated.
- At least one document is available/indexed.
- User submits a non-empty query.

Main Success Scenario:

- User enters a question in chat.
- System validates selected document scope.
- System retrieves relevant chunks from vector index.
- System constructs prompt with retrieved context.
- System calls LLM for response generation.
- System maps sources/citations from retrieved chunks.
- System stores both user and assistant messages.
- System returns answer with citations.

Alternative Scenario:

- No relevant chunks found -> system returns graceful “information not found” response.
- LLM/API call fails -> system returns fallback error message.
- Session/document mismatch -> system blocks request with validation error.

Postcondition:

Answer and source metadata are saved and displayed.

UC Number: UC_QB_06**UC Name: Generate Document Summary**

Primary Actor: User

Secondary Actor: System (RAG Service, LLM Service)

Description: User requests an AI-generated summary of a selected document.

Preconditions:

- User is authenticated.
- Document exists and belongs to user.
- Extracted text is available.

Main Success Scenario:

- User opens a document.
- User selects “Generate Summary.”
- System retrieves high-value context chunks.
- System prepares summary prompt.
- System generates summary through LLM.
- System returns summary to user.

Alternative Scenario:

- Document not found -> system shows not-found message.
- No extractable text -> system returns “No text available to summarize.”
- LLM failure -> system returns temporary error notice.

Postcondition:

Summary is visible and available for export workflow.

UC Number: UC_QB_07

UC Name: Query Multiple Documents

Primary Actor: User

Secondary Actor: System (Session Service, RAG Service, Vector Store, LLM Service)

Description: User asks one question across multiple selected documents for cross-file insight.

Preconditions:

- User is authenticated.
- Two or more user-owned documents are selected.

Main Success Scenario:

- User selects multiple documents.
- User submits a query.
- System validates all document IDs and ownership.
- System retrieves context across selected files.
- System generates unified response from combined evidence.
- System returns answer with multi-source references.
- System stores interaction in session history.

Alternative Scenario:

- Any selected document not owned/found -> request rejected.
- No meaningful context across selected docs -> system returns no-match response.

Postcondition:

Multi-document answer is displayed and persisted in chat history.

UC Number: UC_QB_08

UC Name: View Citations/References

Primary Actor: User

Secondary Actor: System (RAG Service, Metadata Mapper)

Description: User inspects references attached to assistant responses for verification.

Preconditions:

- User is authenticated.
- Assistant response with source metadata exists.

Main Success Scenario:

- User opens citations for a response.
- System fetches source list attached to the message.
- System displays chunk index, page number, and file mapping.
- User verifies response evidence from cited context.

Alternative Scenario:

No source metadata available -> system shows “No citations available for this response.”

Postcondition:

User can validate answer trustworthiness using source references.

UC Number: UC_QB_09

UC Name: Manage Chat History

Primary Actor: User

Secondary Actor: System (Chat Session Service, Database)

Description: User manages conversation history, including list, view, rename, edit, and delete operations.

Preconditions:

- User is authenticated.
- At least one chat session/message exists for management actions.

Main Success Scenario:

- User opens chat history.
- System lists user sessions in reverse chronological order.
- User opens a session to view messages.
- User may rename a session.
- User may edit a previous user message.
- System regenerates assistant response for edited message context.
- User may delete a session when no longer required.

Alternative Scenario:

- Session/message not found -> system returns not-found error.
- Edit attempted on assistant message -> system blocks operation.
- Unauthorized session access -> system denies request.

Postcondition:

Chat history reflects user’s requested updates consistently.

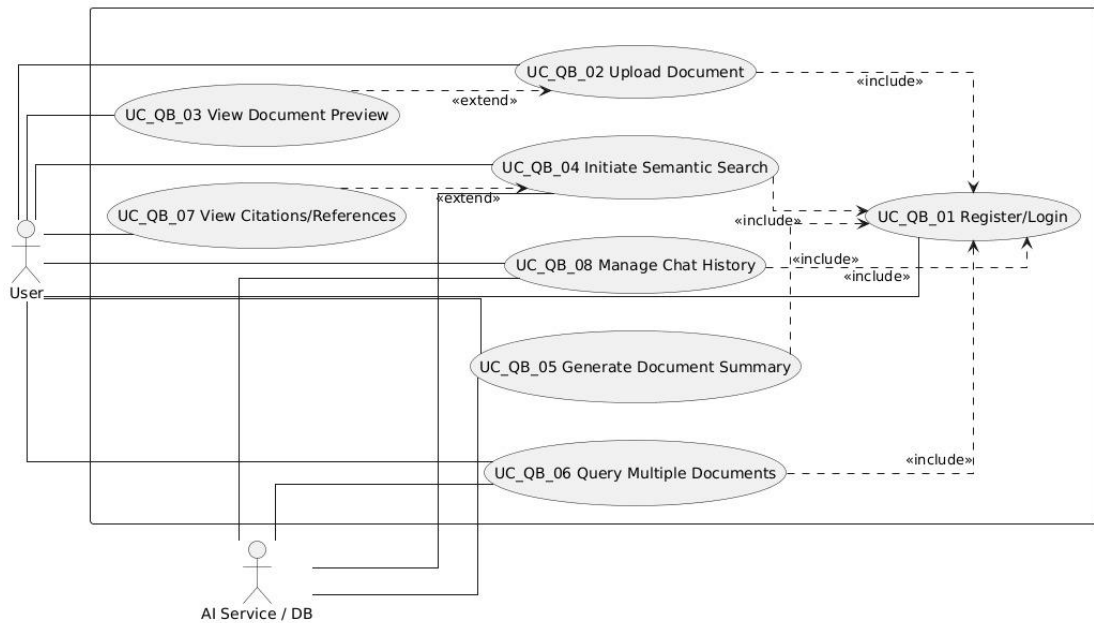


Figure 3.1: User Use Case

CHAPTER 04

DESIGN

4.1 Architecture Diagram

4.1.1 Introduction

QueryBot is designed using a three-tier architecture that separates presentation, application logic, and data management. This structure improves maintainability, supports future scalability, and keeps responsibilities clearly divided across system components.

- **Presentation Tier:** User-facing web interface for authentication, document handling, and conversational interaction.
- **Application Tier:** Backend services that execute authentication, document processing, retrieval, and response generation workflows.
- **Data Tier:** Persistent and semantic storage for transactional records and vectorized document knowledge.

4.1.2 Component Breakdown

1. Frontend (Presentation Layer)

- i. **Technology:** Next.js (React) with Tailwind CSS
- ii. **Responsibilities:** User registration/login screens, document upload workflows, chat interface, citation visibility, summary and export actions.

2. Backend API (Application Layer)

- i. **Technology:** FastAPI (Python)
- ii. **Responsibilities:** API endpoint handling, JWT-based authorization, document parsing and preprocessing, chunking/indexing pipeline, retrieval orchestration, LLM request handling, citation mapping, and chat/session management.

3. Data Layer (Persistence + Retrieval)

- i. **SQLite:** Stores user records, authentication-linked entities, sessions, chat messages, and operational metadata.
- ii. **ChromaDB:** Stores embeddings and chunk metadata for semantic retrieval and relevance matching.

4. AI Services

- i. **LangChain:** Coordinates retrieval and prompt orchestration within the RAG pipeline.
- ii. **Groq (LLM endpoint):** Produces context-aware answers and summaries from retrieved evidence.

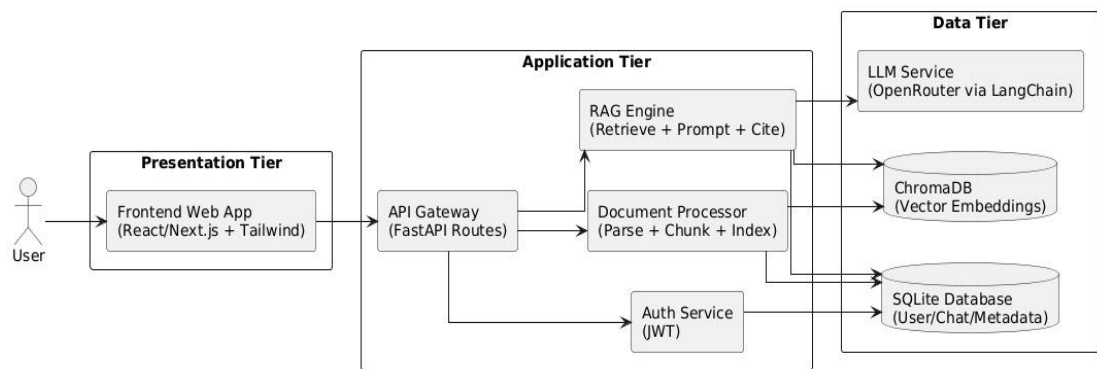


Figure 4.1: Architecture Diagram

4.2 ENTITY RELATIONSHIP DIAGRAM (ERD)

4.2.1 Entities and Attributes

i. Entity: users

- user_id: Unique user identifier (Primary Key, UUID)
- email: User email (Unique)
- hashed_password: Password hash for authentication
- created_at: Account creation timestamp

ii. Entity: documents

- document_id: Unique document identifier (Primary Key, UUID)
- user_id: Owner user ID (Foreign Key → users.user_id)
- original_filename: Original uploaded file name
- stored_path: Server storage path of uploaded file
- mime_type: File MIME type (for example application/pdf)
- size_bytes: File size in bytes
- extracted_text: Parsed textual content
- created_at: Upload timestamp

iii. Entity: chat_sessions

- session_id: Unique session identifier (Primary Key, UUID)
- user_id: Session owner ID (Foreign Key → users.user_id)
- title: Chat session title
- created_at: Session creation timestamp

iv. Entity: chat_session_documents (Junction Table)

- session_id: Session ID (Foreign Key → chat_sessions.session_id, part of composite Primary Key)
- document_id: Document ID (Foreign Key → documents.document_id, part of composite Primary Key)

v. Entity: messages

- message_id: Unique message identifier (Primary Key, UUID)
- session_id: Related chat session ID (Foreign Key → chat_sessions.session_id)
- role: Message role (user / assistant)
- content: Message text
- created_at: Message timestamp

vi. Entity: bookmarks

- bookmark_id: Unique bookmark identifier (Primary Key, UUID)
- user_id: Bookmark owner ID (Foreign Key → users.user_id)
- message_id: Referenced message ID (Foreign Key → messages.message_id)
- snippet: Saved excerpt/snippet
- created_at: Bookmark creation timestamp
- Constraint: Unique pair (user_id, message_id)

4.2.2 Relationships

1. Users → Documents:

One user can upload many documents; each document belongs to one user.
documents.user_id → users.user_id

2. Users → Chat_sessions:

One user can have many chat sessions; each session belongs to one user.
chat_sessions.user_id → users.user_id

3. Chat_sessions ↔ Documents (many-to-many):

Implemented through chat_session_documents.
chat_session_documents.session_id → chat_sessions.session_id
chat_session_documents.document_id → documents.document_id

4. Chat_sessions → Messages:

One chat session can contain many messages; each message belongs to one session.
messages.session_id → chat_sessions.session_id

5. Users → Bookmarks:

One user can create many bookmarks; each bookmark belongs to one user.
bookmarks.user_id → users.user_id

6. Messages → Bookmarks:

One message can be referenced by many bookmarks (from different users); each bookmark points to one message.
bookmarks.message_id → messages.message_id

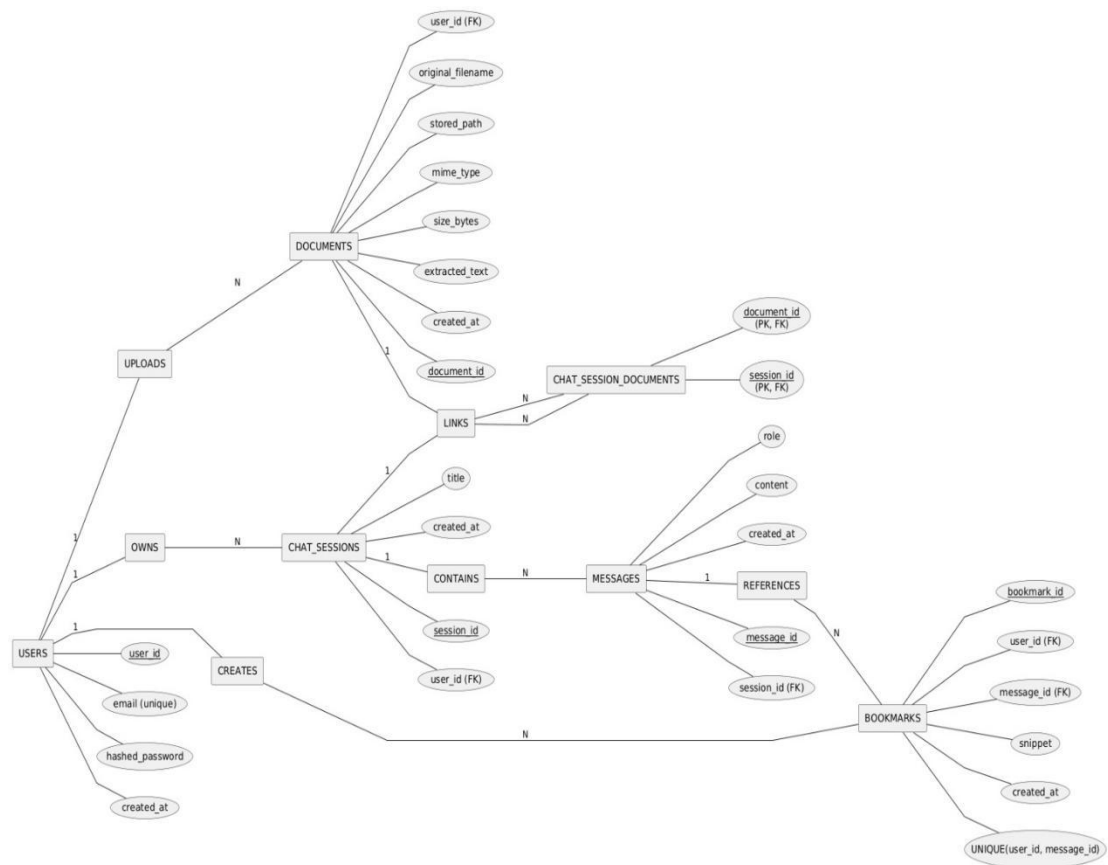


Figure 4.2: Entity Relationship Diagram

4.3 Data Flow Diagrams

4.3.1 Data Flow diagram (Level 0)

The Context Diagram serves as the foundational view of the QueryBot system's data architecture. It establishes the system boundary by representing the entire application—including the Next.js frontend and FastAPI backend—as a single high-level process, identified as Process 0.0. This abstraction is essential for defining how the system interacts with the outside world before detailing internal data handlings.

External Entities and Interactions:

1. **User Interaction:** The User acts as the primary external entity. Data flowing from the User into the system includes authentication credentials for secure access, document files (such as PDF and DOCX) for processing, and natural language queries intended for the AI. Conversely, the system provides the User with essential data outputs, including login/registration status, document upload confirmations, and the final AI-generated responses, summaries, and chat history.
2. **LLM Service Integration:** Since the core AI intelligence is handled externally via Groq, the LLM Service is treated as a secondary external entity. The QueryBot system sends "Augmented Prompts"—which are refined versions of the user's query combined with relevant document context—to the LLM. The service then processes this data and returns generated text responses back to the system for final delivery to the user.

Data Flow Principles:

3. **System Boundary:** At this level, internal data persistence layers like PostgreSQL (for structured data) and ChromaDB (for vector embeddings) are kept within the system boundary and are not illustrated. This ensures that the Context Diagram remains focused strictly on external interfaces and data exchange.
4. **Logical Processing:** The diagram illustrates that every user request undergoes a transformation within the QueryBot boundary, where external knowledge (from documents) and external intelligence (from the LLM) are synthesized to produce the final output.

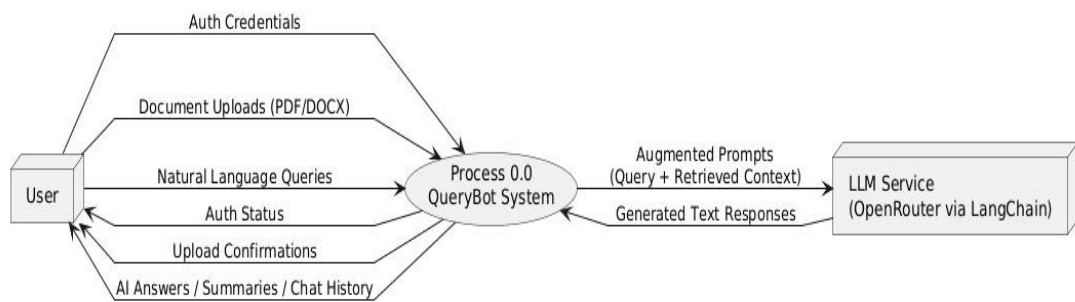


Figure 4.3.1: Level 0 Data Flow Diagram

4.3.2 Data Flow Diagram (Level 1)

Functional Decomposition (QueryBot) DFD Level 1 decomposes Process 0.0 (QueryBot System) into major internal processes to show how data is validated, transformed, stored, retrieved, and returned to the user. At this level, the system behavior is represented through five core processes:

1. User Authentication:

This process handles registration and login requests received from the user. Credentials are validated against stored account data in SQLite, and an authentication result is returned. This step ensures that document and chat operations are performed only within the authorized user scope.

2. Document Upload and Processing:

This process receives uploaded files (PDF, DOCX, and TXT), validates the file format and size, extracts text, and prepares the content for retrieval. After preprocessing, document metadata and extracted text are stored in SQLite, while vector representations are indexed in ChromaDB for semantic search.

3. Semantic Retrieval:

It receives natural-language questions from the user and retrieves relevant context from ChromaDB. The process transforms the user query into a retrieval-ready form and gathers the most relevant document chunks needed for grounded answer generation.

4. AI Response Generation:

This process takes the retrieved context and constructs an augmented prompt. This prompt is sent to the external LLM Service (OpenRouter via LangChain), which returns generated text. QueryBot then structures the returned content into a response-ready output for the application.

5. Chat History and Output Management:

This final process stores user queries and assistant responses in SQLite as persistent session history. It also returns the final outputs to the user, including contextual answers, summaries, and conversation history views.

Summary: In summary, DFD Level 1 shows that QueryBot processes user input through a controlled pipeline: Authenticate $\rightarrow$ Process Document $\rightarrow$ Retrieve Context $\rightarrow$ Generate AI Response $\rightarrow$ Persist and Deliver Output. This decomposition accurately reflects the implemented frontend-backend workflow and the role of SQLite, ChromaDB, and external LLM integration in the project.

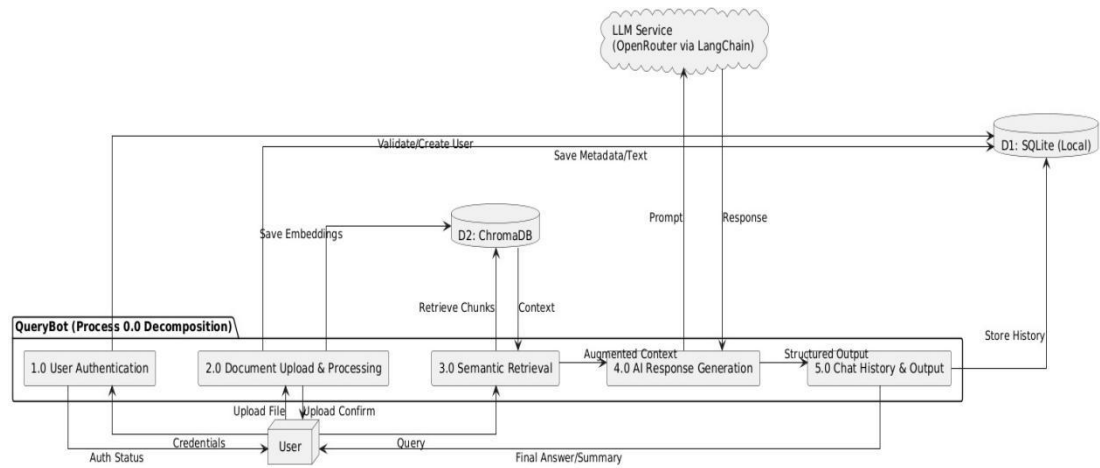


Figure 4.3.2: Level 1 Data Flow Diagram

4.4 CLASS DIAGRAM

4.4.1 Introduction

The Class Diagram defines the static structural model of QueryBot by identifying the core classes, their major attributes, and the relationships among them. It reflects both domain entities (stored application data) and service-layer classes (workflow and processing logic). This model is fully aligned with the implemented Next.js frontend and FastAPI backend architecture.

4.4.2 Core Domain Classes

The core domain model is composed of six primary classes that represent the persistent data of the application:

- i. **User:** Represents a registered account. It stores `id`, `email`, `hashed_password`, and `created_at`. A user can own multiple documents, chat sessions, and bookmarks.
- ii. **Document:** Represents an uploaded file and its processed content. It stores `id`, `user_id`, `original_filename`, `stored_path`, `mime_type`, `size_bytes`, `extracted_text`, and `created_at`. Each document belongs to exactly one user.
- iii. **ChatSession:** Represents a user-specific conversation container. It stores `id`, `user_id`, `title`, and `created_at`. Each chat session belongs to one user and acts as a container for multiple messages.
- iv. **Message:** Represents an individual chat entry within a session. It stores `id`, `session_id`, `role` (user/assistant), `content`, and `created_at`. Every message is linked to a specific chat session.
- v. **Bookmark:** Represents a saved reference to a message for a user. It stores `id`, `user_id`, `message_id`, `snippet`, and `created_at`. A uniqueness constraint on (`user_id`, `message_id`) prevents duplicate bookmarks.
- vi. **ChatSessionDocument:** Represents the associative (junction) class for many-to-many mapping between chat sessions and documents. It stores `session_id` and `document_id`.

4.4.3 Service Classes

Service-level classes encapsulate the business logic and process orchestration of the system:

- i. **AuthService:** Handles user registration, login verification, and JWT token creation.
- ii. **DocumentService:** Manages file upload validation, text extraction flow, preview generation support, and indexing trigger operations.
- iii. **RagService:** Handles retrieval-augmented generation logic, including indexing, semantic retrieval, context building, and LLM-based answer/summary generation.
- iv. **ChatService:** Manages chat session creation, message handling, message

editing/regeneration flow, and persistent chat history management.

4.4.4 Relationship Overview

The structural integrity of QueryBot is maintained through the following key relationships:

- i. **One-to-Many (1:N):** One User can own many Document records and many ChatSession records. Similarly, one ChatSession can contain many Message records.
- ii. **Many-to-Many (M:N):** ChatSession and Document share a many-to-many relationship, managed via the ChatSessionDocument junction class.
- iii. **Referential:** One User can create many Bookmark records, and one Message can be referenced by many bookmarks.
- iv. **Dependency:** Service classes maintain dependency links with domain classes to execute authentication, retrieval, processing, and persistence workflows.

4.4.5 Storage

Relational domain entities are managed in the SQL database layer (SQLite for local development; PostgreSQL for production-oriented architecture). ChromaDB is utilized specifically as a vector store for embeddings and semantic retrieval context, rather than as the primary relational store for domain entities.

4.4.6 Conclusion

The Class Diagram reflects a modular and maintainable system design. The separation of domain entities from service-level orchestration improves architectural clarity, extensibility, and maintainability. The presented model accurately represents the current project scope and technical implementation.

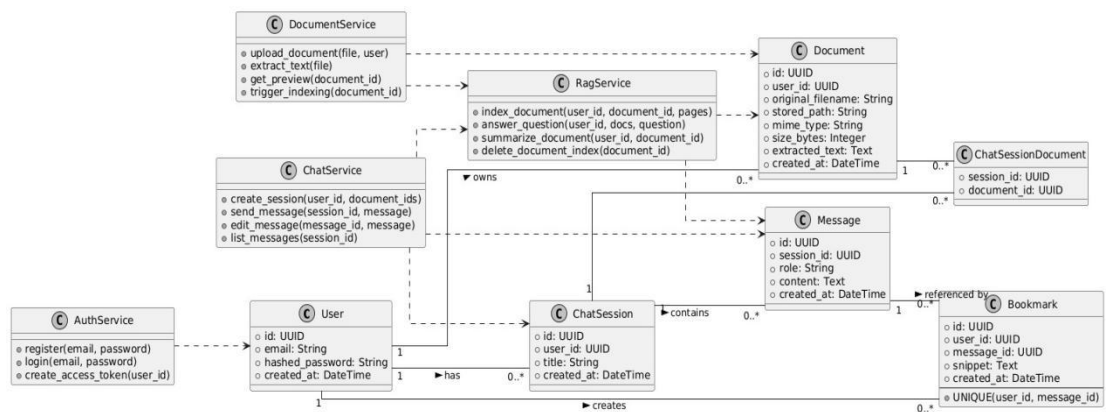


Figure 4.4: Class Diagram

4.5 Sequence Diagram

4.5.1 User Registration and Login Sequence

Purpose: To create a user account and authenticate access using JWT.

Actors / Components: User, Frontend (Next.js), Auth API (FastAPI), Security/Auth , Logic, SQL Database

Sequence Steps:

1. User enters email and password for registration.
2. Frontend sends POST /auth/register request.
3. Auth API checks whether email already exists in SQL Database.
4. If email is new, password is hashed and user record is saved.
5. User enters email and password for login.
6. Frontend sends POST /auth/login request.
7. Auth API verifies credentials against stored hash.
8. API returns JWT access token to Frontend.
9. Frontend stores token and grants access to protected flows.

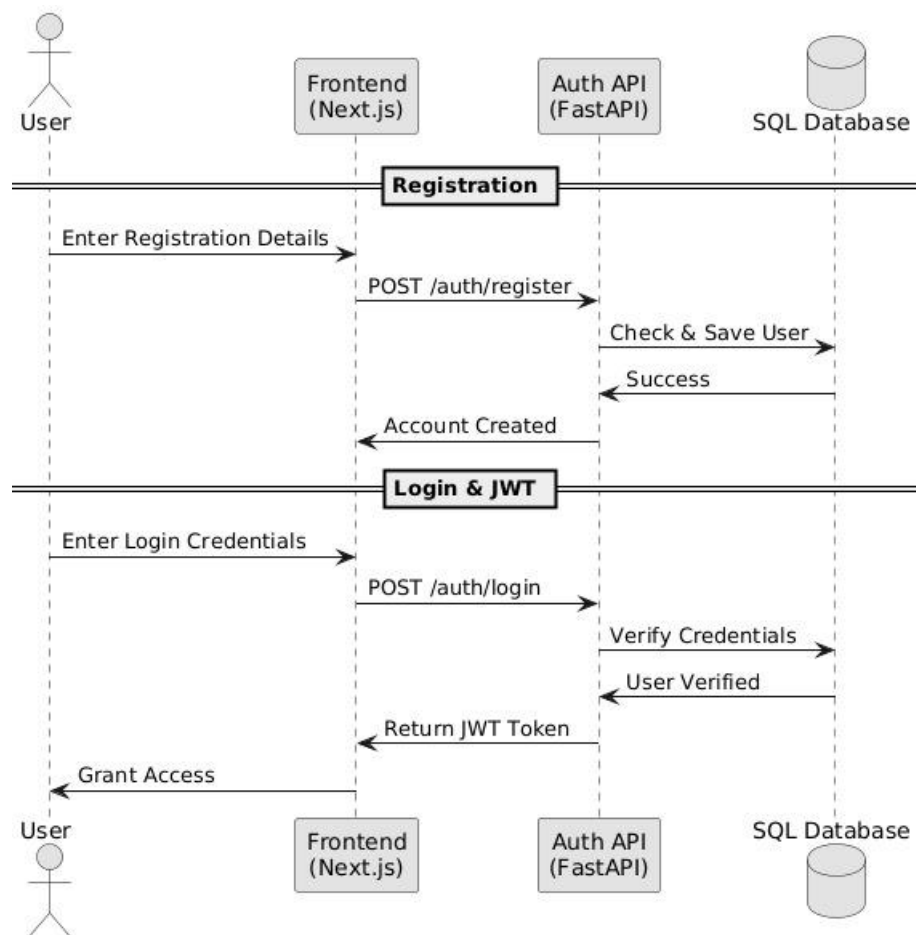


Figure 4.5.1: Sequence diagram for User Registration and Login Sequence

4.5.2. Document Upload and Processing Sequence

Purpose: To upload a supported file and prepare it for retrieval.

Actors / Components:

- User
- Frontend (Next.js)
- Documents API (FastAPI)
- Document Parser/Processor
- RAG Service
- SQL Database
- ChromaDB

Sequence Steps:

- User uploads a file (PDF, DOCX, or TXT).
- Frontend sends POST /documents/upload with auth token.
- API validates file type and size.
- File is stored and text is extracted by parser logic.
- Document metadata + extracted text are saved in SQL Database.
- RAG Service is invoked to chunk and index document content.
- Embeddings/chunks are stored in ChromaDB.
- API returns upload success and document details to Frontend.

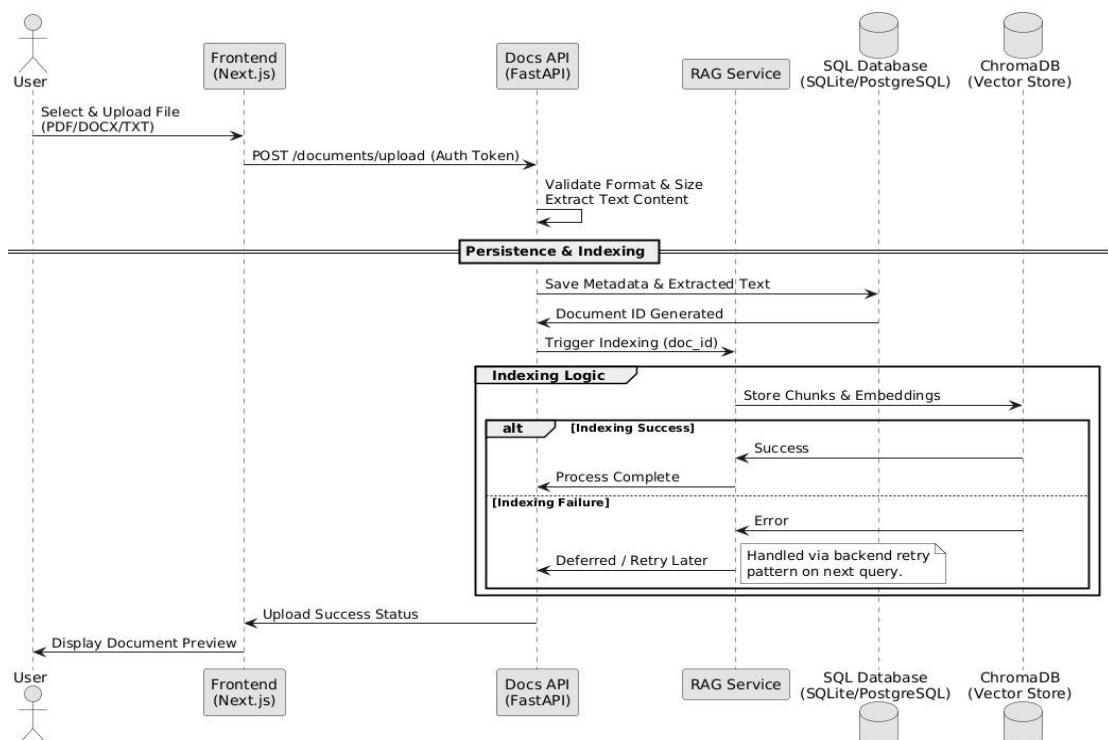


Figure 4.5.2: Sequence diagram for Document Upload and Processing Sequence

4.5.3. Query Processing Sequence (RAG + LLM)

Purpose: To answer a user query using semantic retrieval and LLM generation.

Actors / Components:

- User
- Frontend (Next.js)
- Chat API (FastAPI)
- Chat Orchestration Logic
- RAG Service
- ChromaDB
- LLM Service (Groq via LangChain-compatible client)
- SQL Database

Sequence Steps:

- User submits a natural-language question with selected document scope.
- Frontend sends POST /chat/messages with auth token.
- Chat API validates session/document ownership.
- User message is staged/persisted in SQL Database.
- RAG Service retrieves relevant chunks from ChromaDB.
- RAG Service builds augmented prompt (question + context).
- Prompt is sent to LLM service and response is received.
- Assistant response is saved in SQL Database.
- API returns assistant message plus assistant_sources (citations).
- Frontend displays answer and source references.

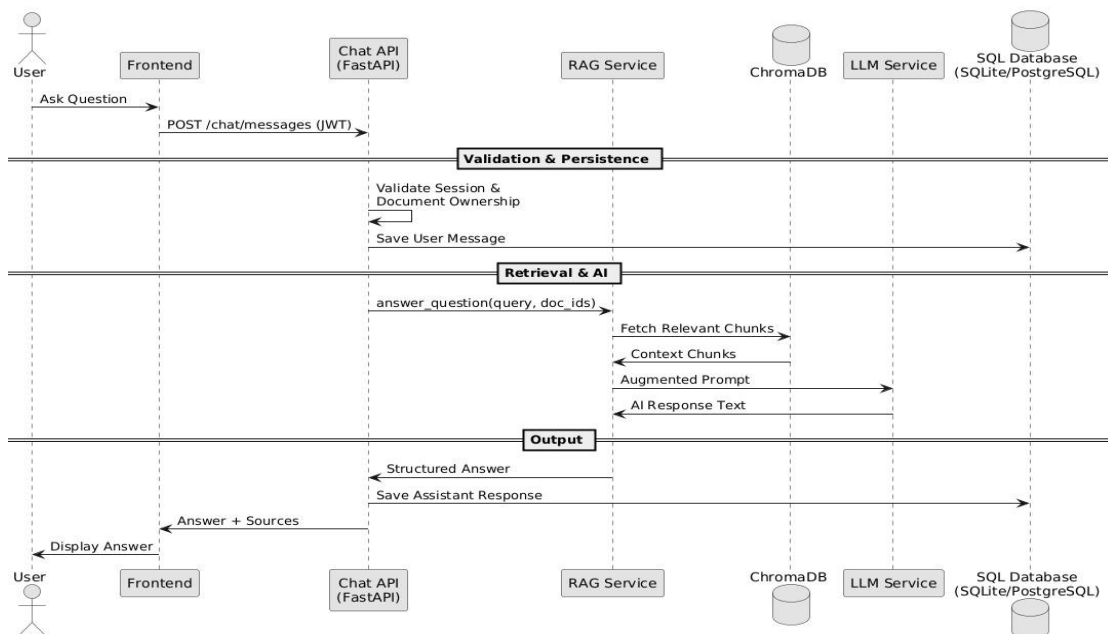


Figure 4.5.3: Sequence diagram for Query Processing Sequence (RAG + LLM)

4.5.4. Document Summary Generation Sequence

Purpose: To generate a contextual summary for a selected document.

Actors / Components:

- User
- Frontend (Next.js)
- Documents API (FastAPI)
- RAG Service
- ChromaDB
- LLM Service

Sequence Steps:

- User requests summary for a document.
- Frontend sends POST /documents/{document_id}/summary.
- API validates that document belongs to authenticated user.
- RAG Service retrieves summary-relevant chunks from ChromaDB.
- RAG Service creates summary prompt and calls LLM service.
- LLM returns summary text.
- API returns summary payload to Frontend.
- Frontend displays summary to user.

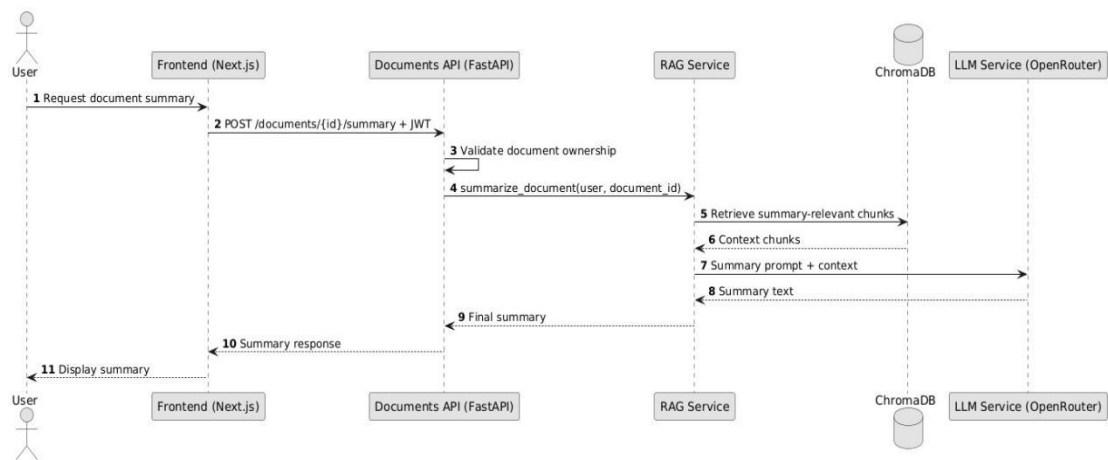


Figure 4.5.4: Sequence diagram for Document Summary Generation Sequence

CHAPTER 05

GRAPHICAL USER INTERFACES

5.1 Introduction

Chapter 5 presents the Graphical User Interface (GUI) of QueryBot and demonstrates how users interact with the system modules implemented in previous chapters. The interface is designed to support an end-to-end workflow, starting from secure authentication and continuing to document upload, AI-powered querying, summary generation, and history-based interaction. This chapter focuses on real implemented screens, validation behavior, navigation flow, and output presentation to show the practical usability of the system.

5.2 Overall Interface Structure

QueryBot follows a web-based interface architecture implemented with Next.js and Tailwind CSS. The interface is organized around task-focused screens that guide the user through authentication, document ingestion, and AI interaction. Navigation is designed to be straightforward, with clear transitions between login/register, document management, chat interaction, summary generation, and bookmark/history views.

The overall structure prioritizes usability and readability by using consistent visual components, validation feedback near input fields, and action-oriented controls. Since the current project scope includes only end-user functionality, the interface does not include any administrative dashboard.

5.3 Core Interface Screens

5.3.1 Login Screen

The Login Screen serves as the primary entry point for existing users. It provides dedicated input fields for email and password, along with a password visibility toggle to improve usability. The interface includes real-time inline validation, where invalid email format and non-compliant password input are immediately shown below their respective fields. After successful credential verification, the system issues JWT-based authentication and redirects the user to the protected document workspace.

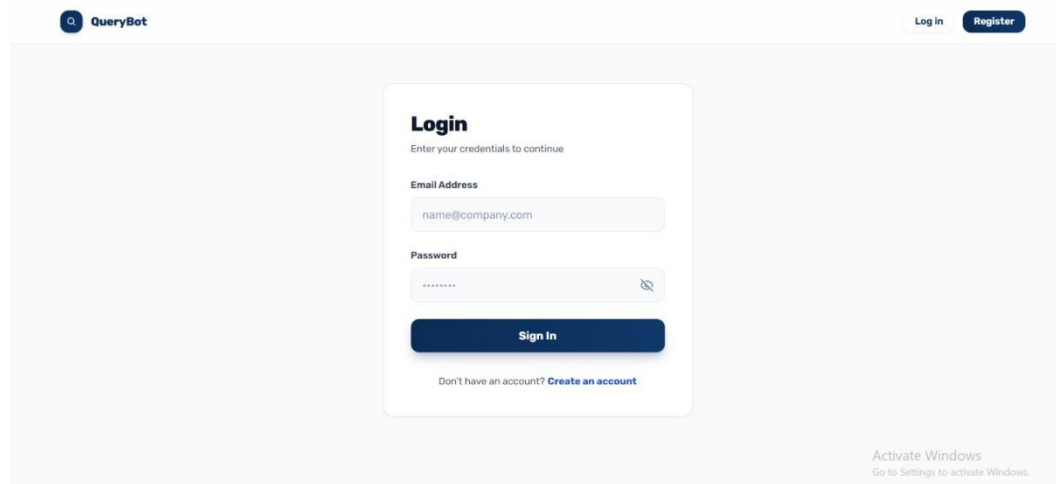


Figure 5.3.1: Login Screen

5.3.2 Registration Screen

The Registration Screen is designed for new users to create an account and start using QueryBot. It includes input fields for email and password, a password visibility toggle, and a terms acceptance checkbox to ensure consent before account creation. The interface applies real-time inline validation, where invalid email format and non-compliant password values are immediately shown below the relevant fields.

After successful registration, the system automatically authenticates the user and redirects to the document workspace, creating a smooth onboarding experience.

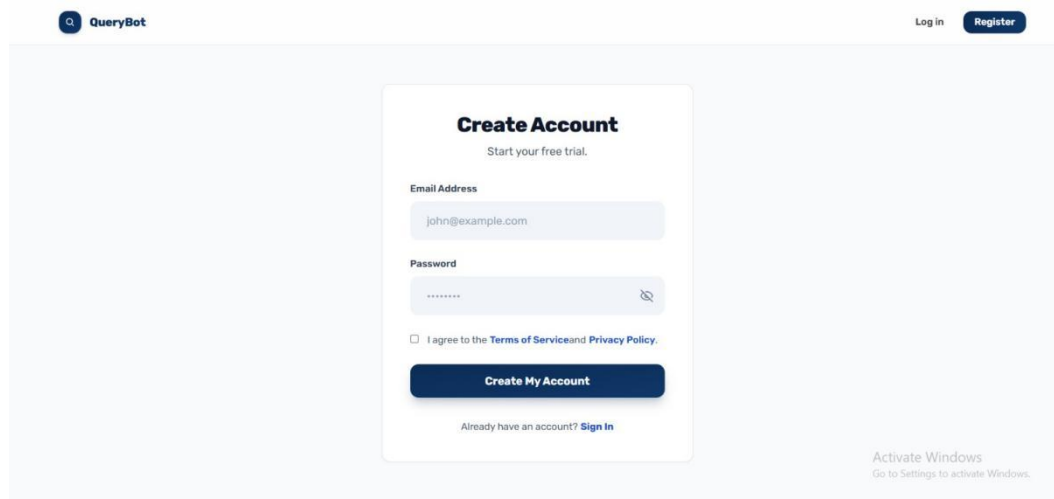
The image shows a web browser window displaying the QueryBot registration interface. At the top left is the QueryBot logo, and at the top right are 'Log in' and 'Register' buttons. The main content is a 'Create Account' form with the subtitle 'Start your free trial.' The form contains an 'Email Address' field with the placeholder 'john@example.com', a 'Password' field with a visibility toggle icon, and a checkbox for 'I agree to the Terms of Service and Privacy Policy.' Below these is a prominent 'Create My Account' button. At the bottom of the form, it says 'Already have an account? Sign in'. In the bottom right corner of the page, there is a 'Activate Windows' watermark with the text 'Go to Settings to activate Windows.'

Figure 5.3.2: Registration Screen

5.3.3 Home Page

The Home Page acts as the initial landing interface of QueryBot and provides a clear entry point to the platform's core features. It introduces the system purpose and guides users toward authentication and document interaction workflows. The page design emphasizes clarity, branding consistency, and quick access to primary navigation actions.

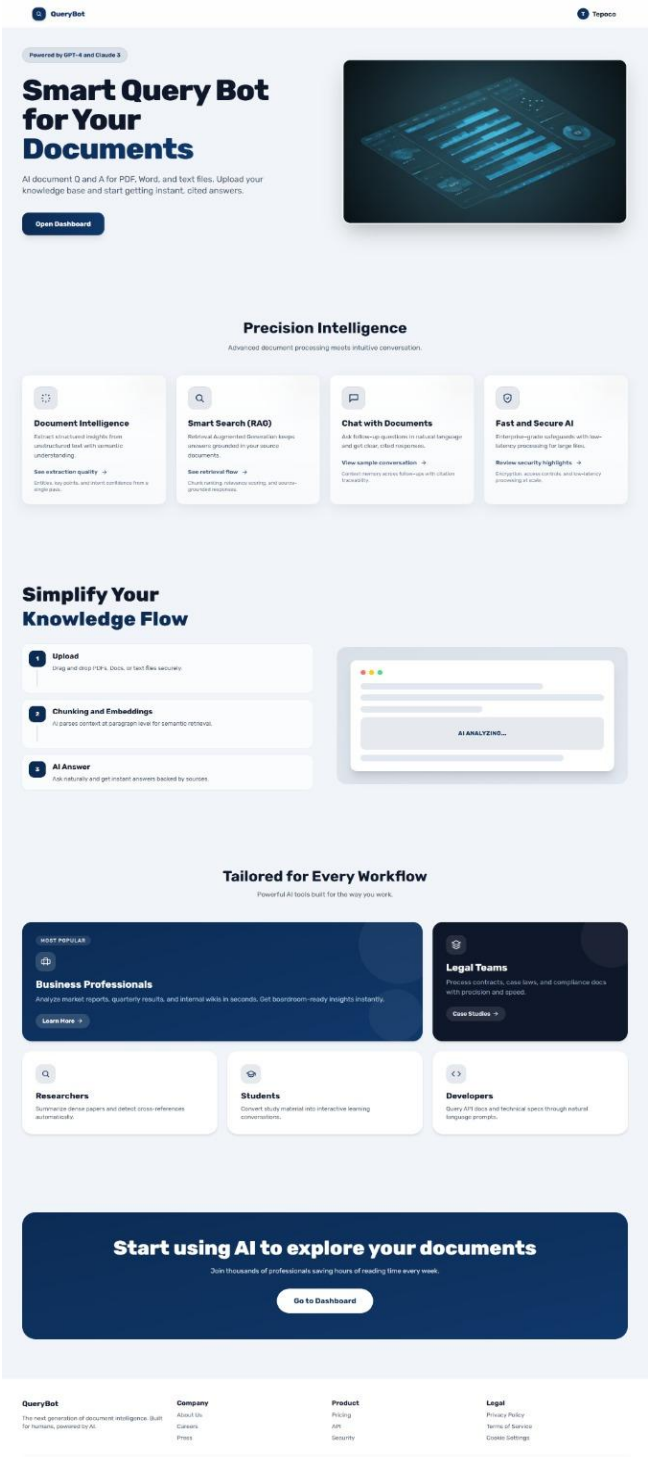


Figure 5.3.3: QueryBot Home Page

5.3.4 Documents Dashboard and Upload Screen

The Documents Dashboard serves as the main workspace after authentication. It displays the user’s uploaded files and provides actions to upload, view, and manage documents. The upload interface accepts supported formats (PDF, DOCX, TXT) and provides status feedback after successful processing. Once a file is uploaded, the system stores metadata, extracts text content, and prepares

the document for semantic querying through the backend retrieval pipeline. This screen acts as the operational entry point for chat, summary generation, and document-level interactions.

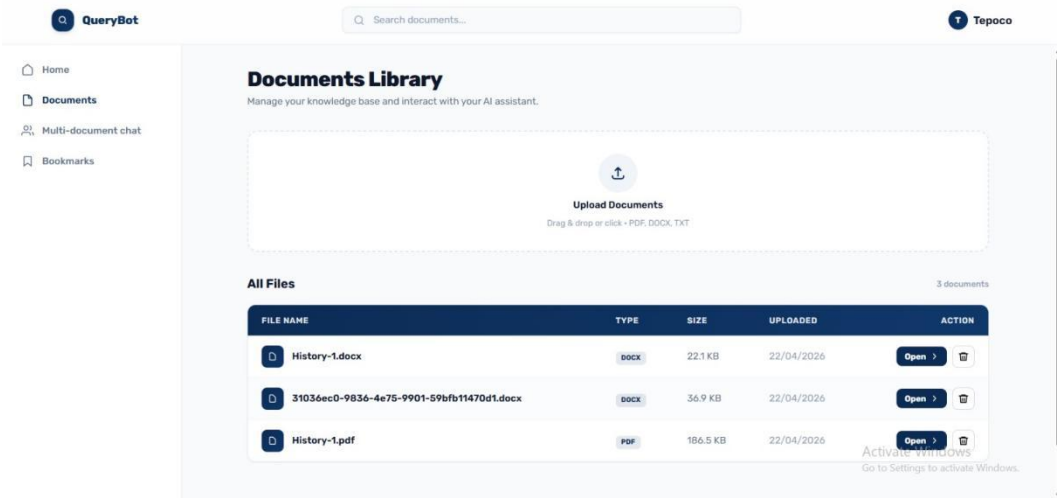


Figure 5.3.4: Documents Dashboard and Upload Screen

5.3.5 Document Upload Progress State

This screen displays the live upload status after a file is selected. It confirms that document transfer and processing are in progress, helping users avoid repeated submissions. After completion, the file appears in the documents table for further actions.

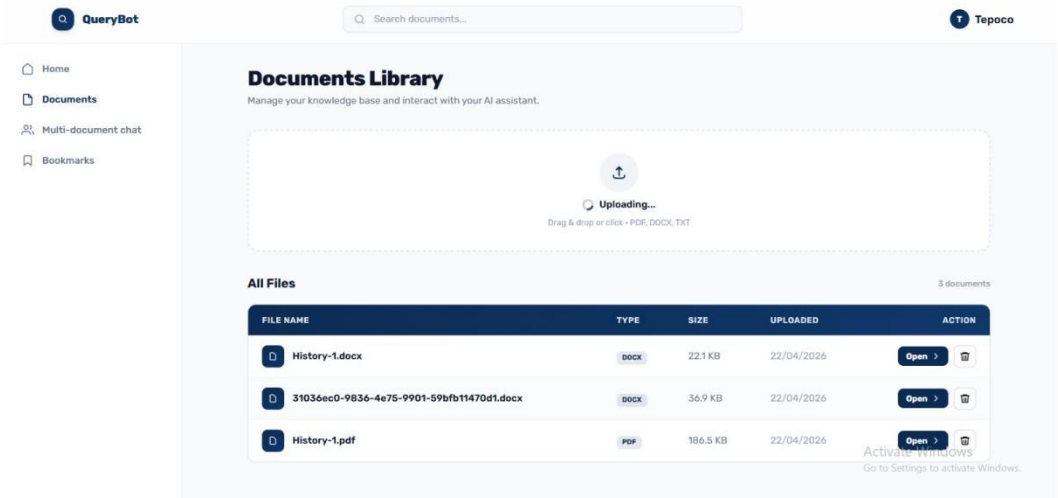


Figure 5.3.5: Document Upload Progress State

5.3.6 Delete Document Action

This screen demonstrates the document deletion feature available in the Documents Dashboard. The user can remove an uploaded file directly from the document list using the delete action. After successful deletion, the document is removed from the table and becomes unavailable for preview, querying, or summarization.

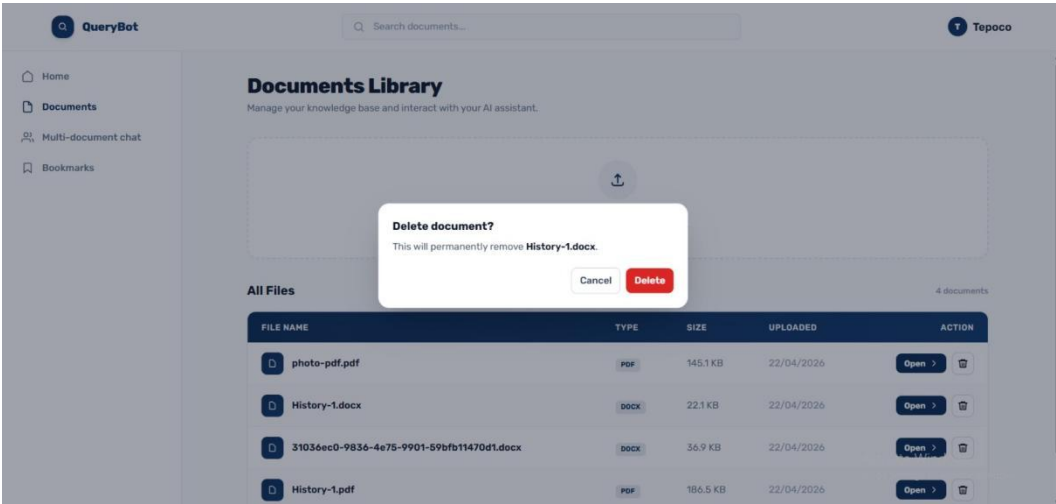


Figure 5.3.6: Delete Document Action in Documents Dashboard

5.3.7 Single-Document Chat Screen

This screen allows the user to ask questions about one selected document and receive context-aware AI responses. The interface presents the conversation flow in message format and supports source-backed answers for improved reliability. It acts as the primary interaction area for document-level querying and response review.

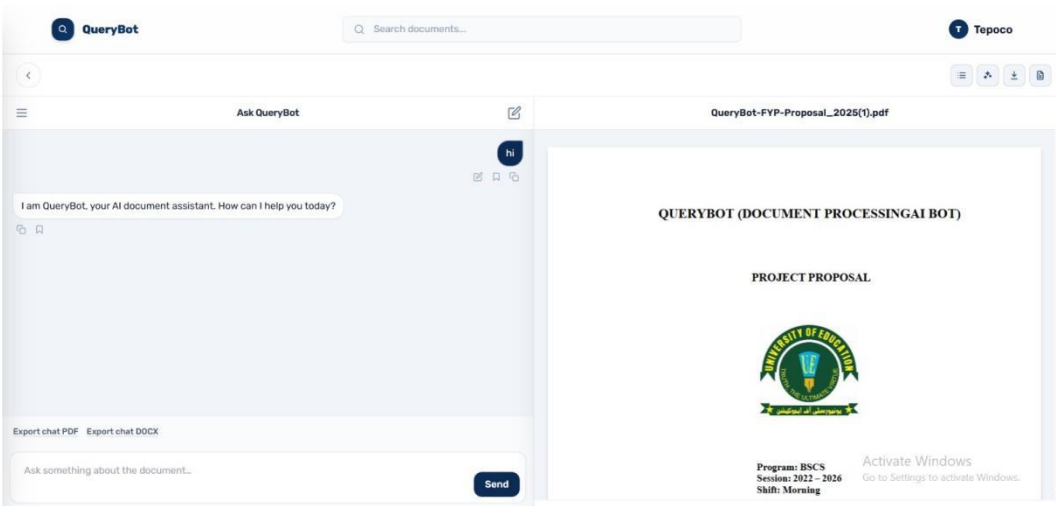


Figure 5.3.7: Single-Document Chat Screen

5.3.8 Edit Message and Regenerate Response

This screen highlights the message editing feature within the single-document chat interface. The user can modify a previously submitted query and trigger regeneration of the assistant response. The updated answer is then displayed in the conversation, allowing users to refine prompts and improve response relevance without starting a new session.

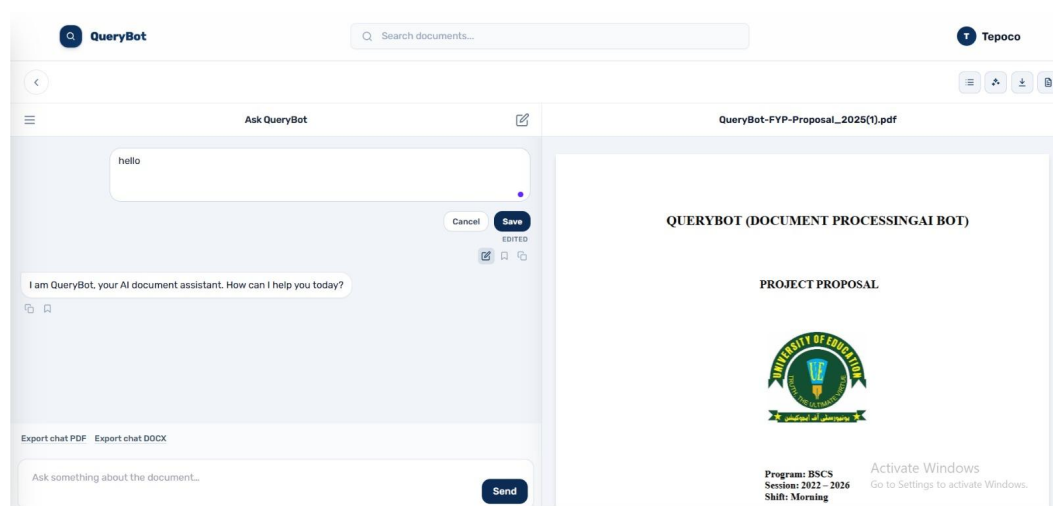


Figure 5.3.8: Edit Message and Regenerate Response Screen

5.3.9 Chat History, Rename, and Delete Session

This screen presents the chat history panel integrated into the chat interface. It allows users to view recent chat sessions, search previous conversations, and continue context-aware interaction from earlier sessions. In addition, the interface provides session management actions, including Rename and Delete, enabling users to organize conversation titles and remove unnecessary chats. This improves usability and keeps the chat workspace structured for long-term usage.

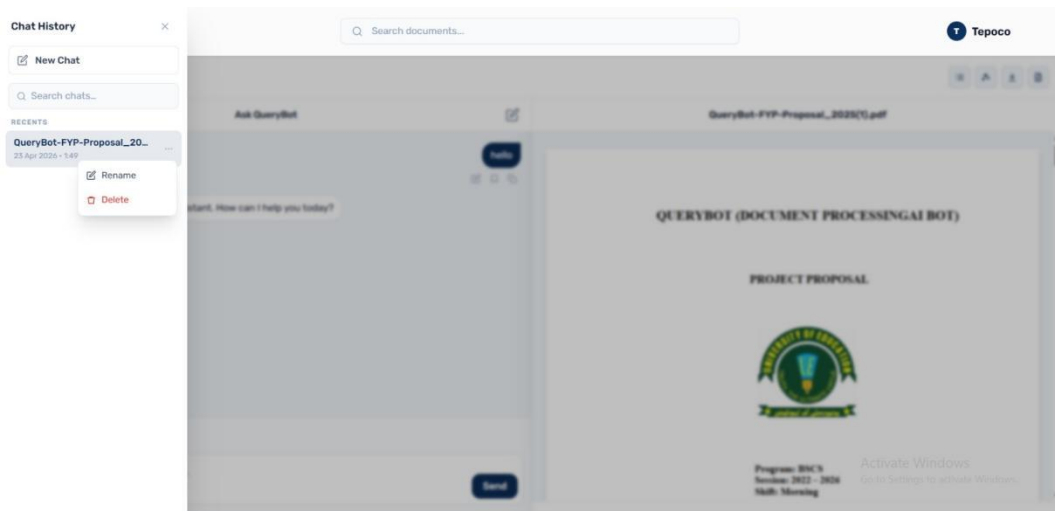


Figure 5.3.9: Chat History Panel with Rename and Delete Actions

5.3.10 Document Search Functionality

This screen demonstrates the document search feature available in the Documents Dashboard. Users can enter keywords in the search bar to quickly filter and locate relevant files from the uploaded document list. Matching results are shown immediately in a dropdown and reflected in the table view, improving navigation efficiency and reducing manual browsing effort.

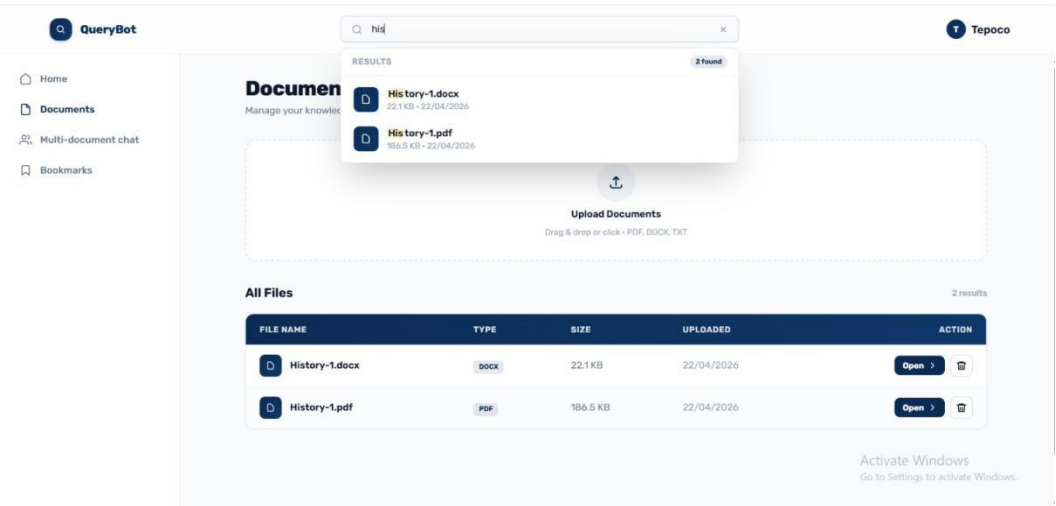


Figure 5.3.10: Document Search Functionality in Document Dashboard

5.3.11 AI Response Thinking State

This screen shows the temporary loading state after a user submits a query in chat. It indicates that the system is processing the request and generating an AI response. This feedback improves user experience by confirming that the request is active.

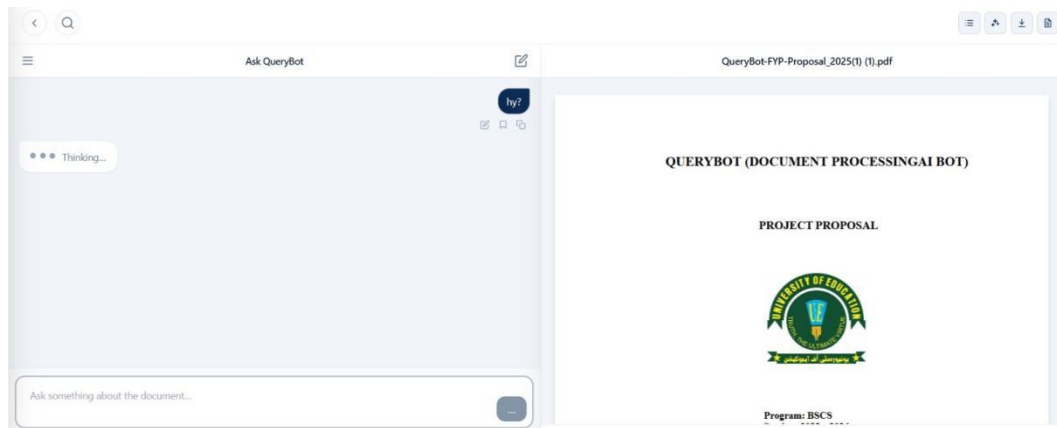


Figure 5.3.11: AI Response Thinking (Loading) State

5.3.12 Live Document Preview

Short content: The Live Document Preview panel displays the selected file alongside the chat interface, allowing users to read source content while asking questions. This side-by-side view improves context understanding and supports citation-based verification during interaction.

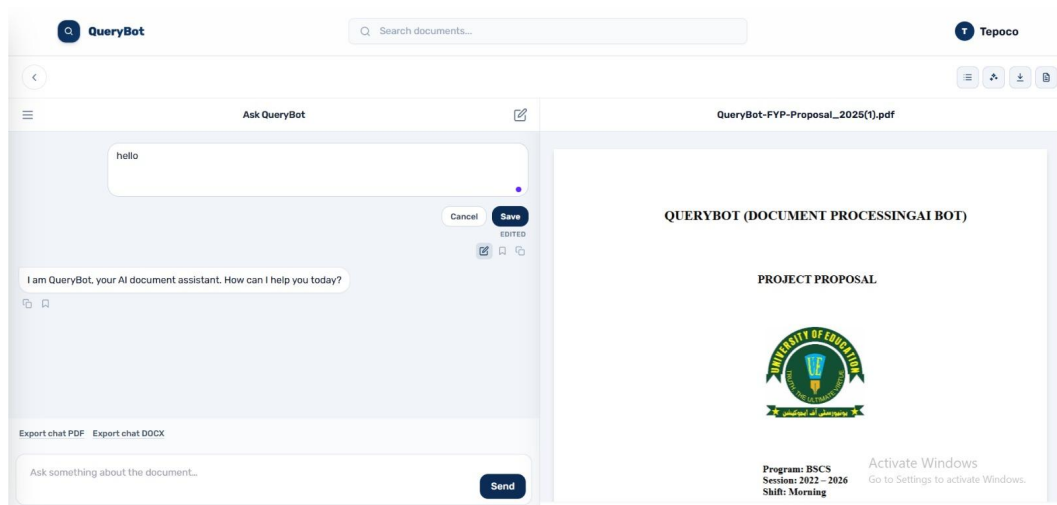


Figure 5.3.12: Live Document Preview Panel

5.3.13 Generate Summary Feature

This screen demonstrates the summary generation feature, where users can request an AI-generated summary of the selected document. The system processes document context through the retrieval pipeline and returns a concise structured output. This helps users quickly understand key information without reading the entire file.

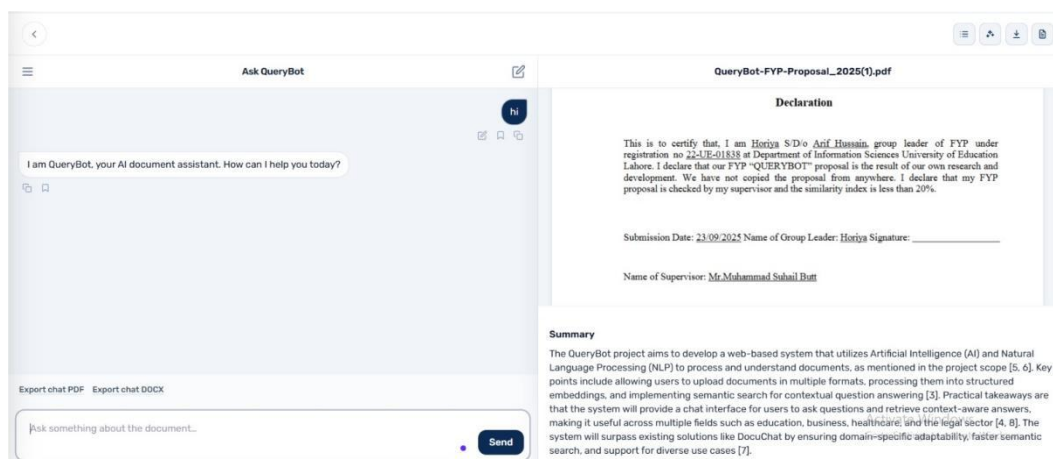


Figure 5.3.13: Generate Summary Feature in Chat Interface

5.3.14 Summary and Export Controls

This screen presents the summary and export controls available in the chat interface. Users can generate a document summary through the AI pipeline and export outputs in PDF or DOCX format for reporting and documentation purposes. These actions provide quick result reuse and improve practical usability for academic workflows.

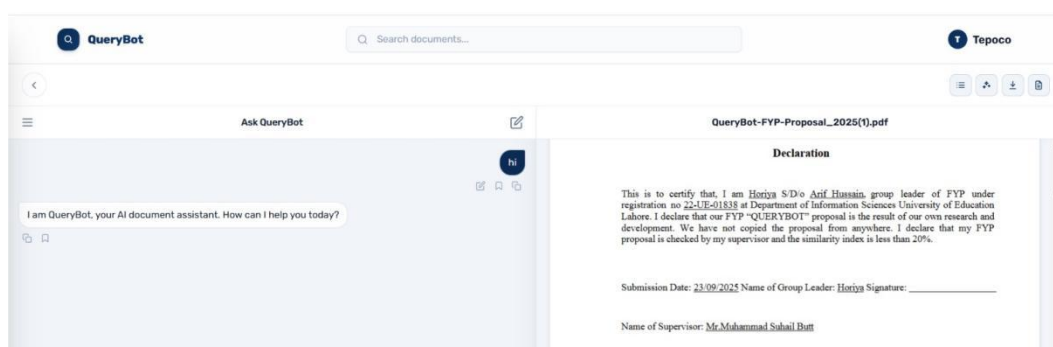


Figure 5.3.14: Generate Summary and Export (PDF/DOCX) Controls

5.3.15 Multi-Document Chat and Multi-File Preview

This screen demonstrates the multi-document chat feature, where users can select multiple uploaded files and submit a single query across all selected sources. The interface also provides preview support for multiple documents, allowing users to inspect file content while interacting with the chat. This combined view improves cross-document analysis by enabling context-aware questioning and quick source verification within the same workflow.

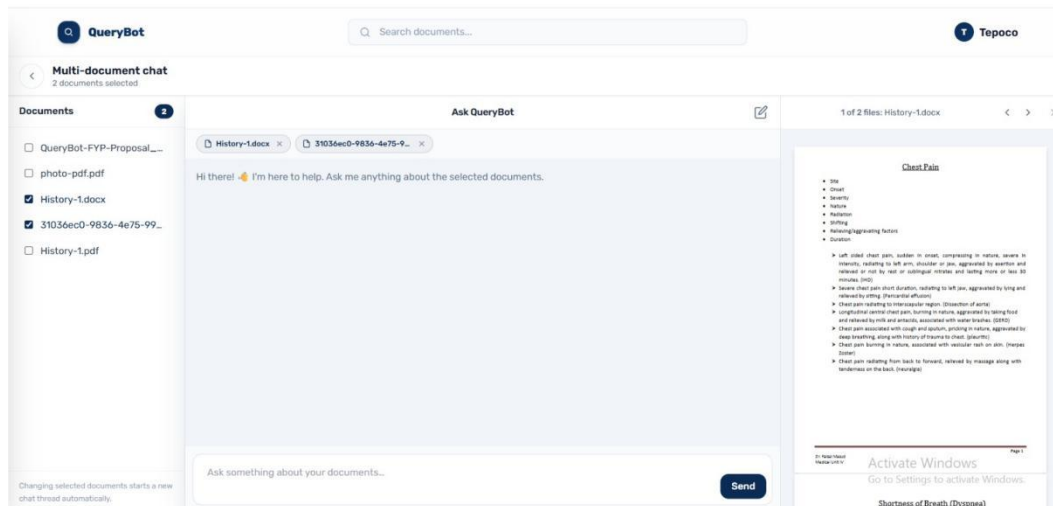


Figure 5.3.15: Multi-Document Chat with Multi-File Preview

5.3.16 Bookmarks Management

This screen presents the bookmark module, where users can save important responses for quick access. Each bookmark can be used to open the related chat session and continue context-aware discussion. Users can also delete bookmarks when they are no longer needed, helping keep saved references organized.

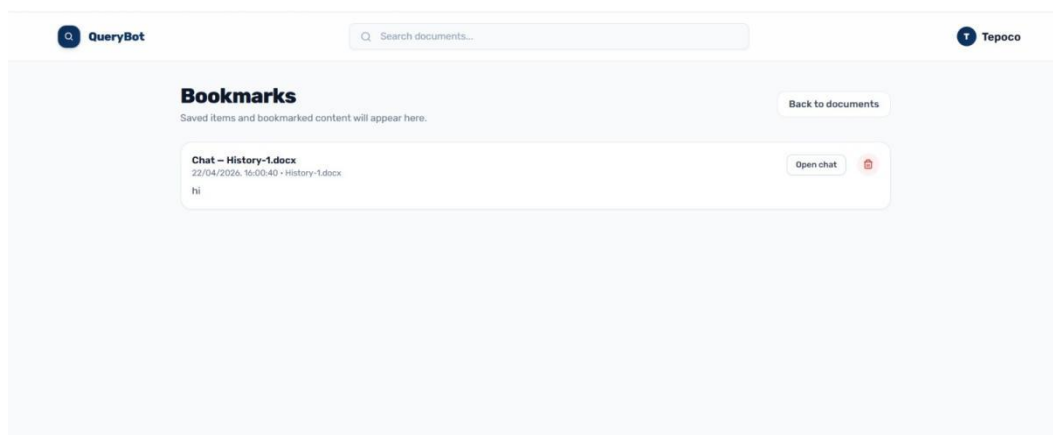


Figure 5.3.16: Bookmarks Screen with Open Chat and Delete Actions

5.3.17 User Profile and Logout Controls

This screen shows the user profile dropdown in the top navigation bar. It displays the signed-in user identity and provides quick access to account-related actions. The Log out option allows users to securely end the current session and return to the authentication flow.

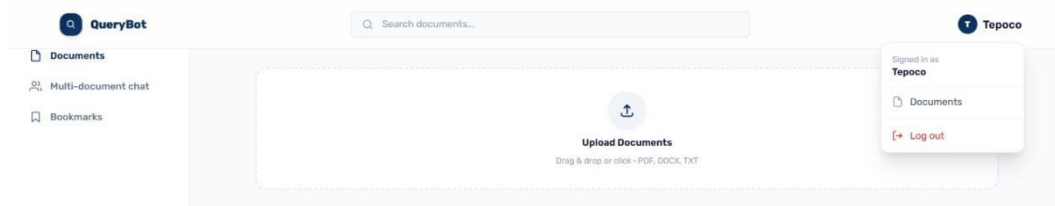


Figure 5.3.17: User Profile Menu and Logout Action

5.3.18 Citations and References View

This screen shows the citation feature in the chat interface, where AI responses are linked with source context from uploaded documents. Users can review referenced chunks/pages to verify answer authenticity and improve trust in generated output. The citation view supports evidence-based interaction and document-grounded response validation.

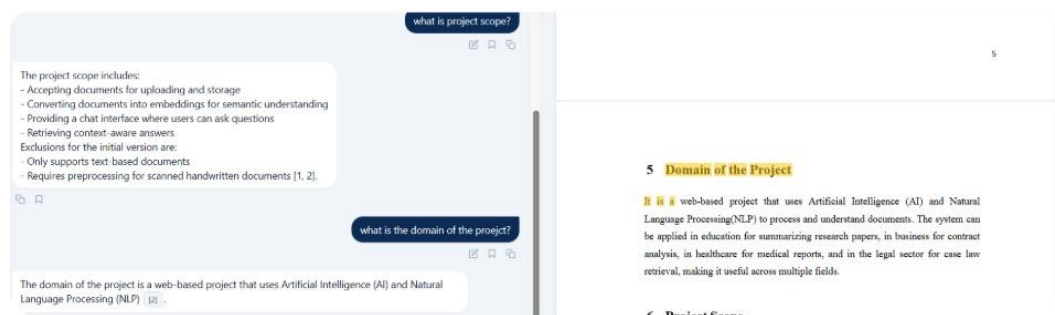


Figure 5.3.18: Citations and References in Chat Response

5.4 Navigation And Workflow

5.4.1 Linear Navigation Model

QueryBot is organized around a linear, task-driven navigation model that connects authentication, document management, and AI interaction within the same web application. After successful login or registration, the user is routed to the Documents Workspace. **In this area:**

- Uploaded files are listed for quick access.
- New documents can be added through a dedicated upload area.
- Users can search for specific documents, open files for interaction, or delete documents when they are no longer required.

5.4.2 Document-Centric AI Workflow

For individual document interaction, the system provides a split workflow designed for productivity:

- i. **Split Interface:** A dual-pane view with the chat interface on one side and a live

document preview on the other.

- ii. **AI Interaction:** Within the chat, users can ask questions and observe real-time "thinking" or loading feedback during response generation.
- iii. **Advanced Features:** Users can review citations and references returned with answers, edit previous messages to regenerate responses, and access full chat history (including session renaming and deletion).
- iv. **Output Management:** The workflow supports generating document summaries and exporting outputs as PDF/DOCX formats.

5.4.3 Multi-Document Analysis

For cross-document analysis, the application provides a specialized navigation path:

- The user navigates to the Multi-document Chat section.
- Multiple files can be selected simultaneously for a single query.
- The RAG engine retrieves context across all chosen documents, with preview support for reviewing multiple sources during the interaction.

5.4.4 Supporting Navigation and Session Management

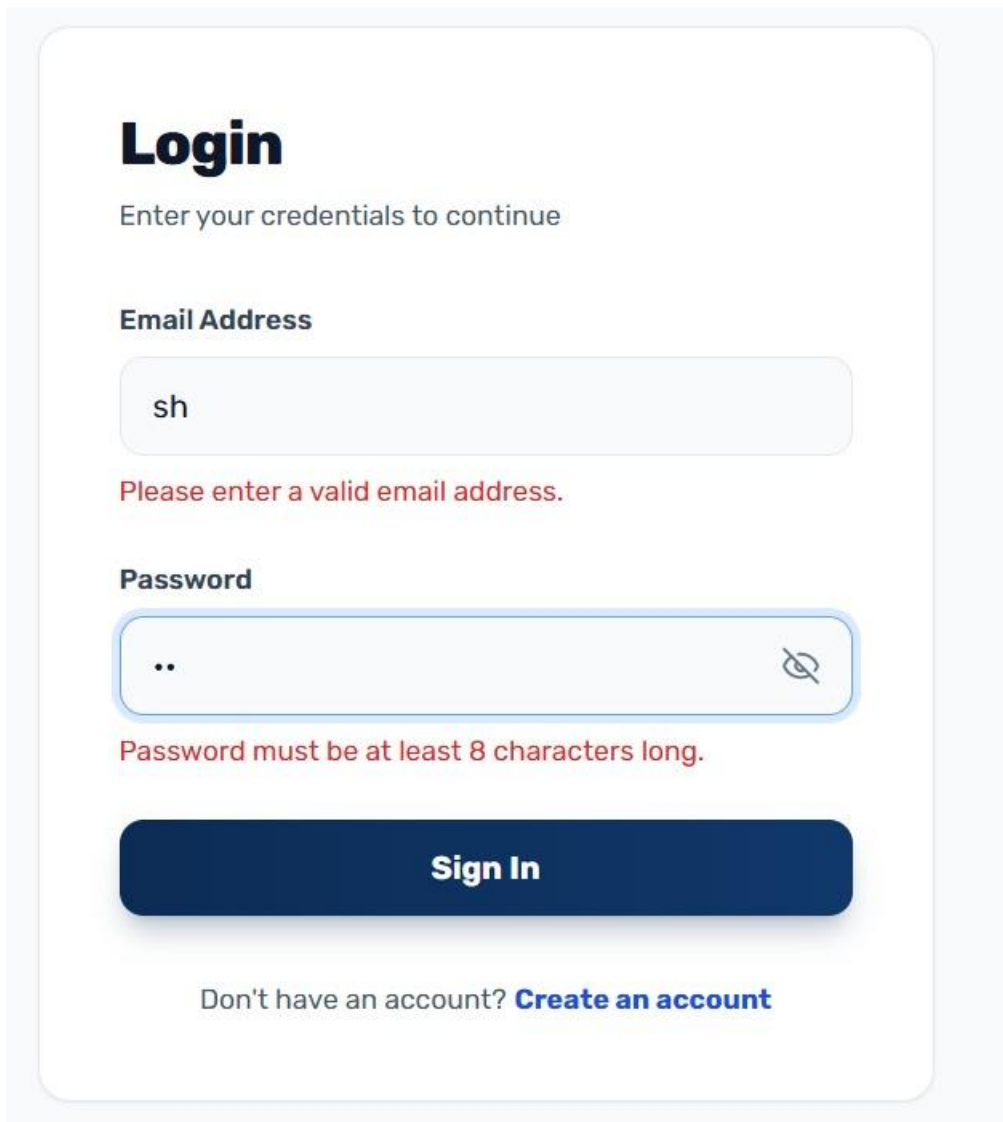
The journey is rounded out by supporting features that enhance user experience:

- i. **Bookmarks:** Users can save important AI responses for quick retrieval and return to the related chat sessions later.
- ii. **User Profile Menu:** Provides session awareness and logout functionality.
- iii. **Coherent Journey:** The overall workflow is designed to minimize context switching between unrelated screens, keeping upload, retrieval, answering, summarization, and export actions seamlessly connected.

5.5 Form Validation and Error Messages (UI focus)

5.5.1 Login Form Inline Validation

The Login screen validates user input before submission. If the email format is invalid, a message appears directly under the email field. If the password does not meet the minimum length rule, a message appears under the password field. This inline feedback helps users correct mistakes immediately without waiting for a server response.



Login

Enter your credentials to continue

Email Address

sh

Please enter a valid email address.

Password

..

Password must be at least 8 characters long.

Sign In

Don't have an account? [Create an account](#)

Figure 5.5.1: Login Form Inline Validation (Email and Password)

5.5.2 Document Upload Validation (Type and Size)

The Documents upload area validates files before processing. If the user selects an unsupported format, the UI shows an error such as “Invalid file type” and lists the allowed extensions (PDF, DOCX, TXT). If the file exceeds the configured limit, the UI shows “File too large” with the maximum size (10 MB). Both checks use the same inline error banner on the Documents screen so the user can correct the file and retry without leaving the page.

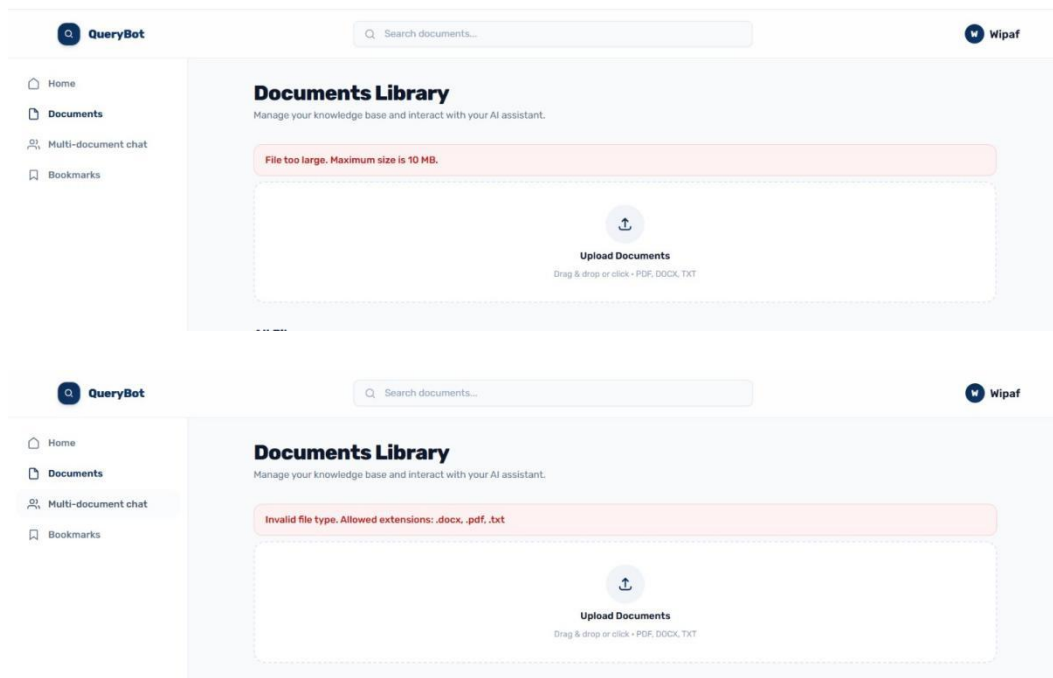


Figure 5.5.2: Document Upload Validation Invalid Type and Oversized File

CHAPTER 06

TESTING

6.1 Introduction

This chapter documents the testing of QueryBot to verify that the implemented system behaves as required and delivers a reliable user experience. Testing focuses on validating core functional flows, including user authentication, document upload and management, chat-based question answering, summary generation, and supporting features such as bookmarks and exports. The results presented in this chapter are based on manual, black-box testing performed on the running application, where test cases are defined from a user's perspective and outcomes are compared against expected behavior without inspecting internal code paths.

6.2 Testing Strategy

Testing for QueryBot is carried out using a manual black-box approach. Test cases are designed from the end-user viewpoint: inputs are provided through the web interface, and results are evaluated by observing system responses, UI updates, and output correctness. This strategy is appropriate for validating integrated behavior across the Next.js frontend and FastAPI backend, including authentication, file handling, retrieval-based responses, and persistent chat data. Testing is performed in a local development environment with a stable network connection, and each test is repeated when fixes are applied to confirm that the issue is resolved.

6.3 Types of Testing Performed

The following testing types were applied during the QueryBot evaluation. These categories describe how the system was validated, while detailed evidence is recorded later in the test case section.

1. Functional (Black-Box) Testing

This type verifies that the main software features behave correctly from a user perspective. Testers exercise complete workflows such as registration, login, document upload, chat-based questioning, summary generation, export actions, and bookmark management. The focus is on expected outcomes and system responses rather than internal implementation details.

2. UI/Usability Testing

This type evaluates the clarity and consistency of the interface. It includes navigation between pages, readability of labels and messages, and the visibility of feedback such as inline validation text, error banners, and loading indicators. The goal is to ensure the application is easy to understand and operate during normal use.

3. Input Validation Testing

This type confirms that user inputs are checked before processing. It includes email and password rules on authentication screens, and upload restrictions such as supported file types and maximum file size on the Documents page. Validation testing helps prevent invalid data from reaching backend processing and

improves overall data quality.

4. Integration Testing (Light)

This type checks selected end-to-end paths where the frontend, backend API, and data handling work together. For example, after uploading a document, the user can query the same document and export results, confirming that upload, retrieval, and output features remain consistent within one continuous workflow.

6.4 Test Plan

6.4.1 Features to Be Tested

The following core features of QueryBot were selected for testing:

- User registration and login authentication
- Protected route access (unauthenticated users redirected to login)
- Document upload with supported file type checks (.pdf, .docx, .txt)
- Upload size validation (maximum 10 MB per file)
- Document listing, search, open, and delete operations
- Single-document chat flow (ask question, get AI response, citations)
- Multi-document chat flow with document selection and session continuity
- Message editing and regeneration of assistant response
- Bookmark add/remove and bookmark listing
- Summary generation for selected document
- Export of summary and chat to PDF and DOCX formats
- Error handling for invalid input and failed API operations

6.4.2 Testing Approach

Testing was primarily performed using manual black-box testing with selected integration checks:

1. **Functional checks:** End-to-end user workflows from UI actions to backend response.
2. **Validation checks:** Invalid email/password, unsupported file types, oversized files, and wrong export format.
3. **Integration checks:** Frontend + FastAPI backend + SQLite + vector indexing flow for upload/chat/summary/export.
4. **Regression checks:** After changes in chat, upload, or bookmarks, core workflows were re-tested to ensure no feature breakage.

6.4.3 Testing Environment

Hardware

Development laptop/PC (8GB+ RAM, SSD, standard CPU)

Software Stack

Frontend: Next.js (React + TypeScript)

Backend: FastAPI (Python)

Database: SQLite (querybot.db)

Vector store: ChromaDB

Runtime: Node.js and Python virtual environment

Browser

Google Chrome (primary)

Microsoft Edge (cross-check)

Operating System

Windows 10/11 environment (project executed locally)

6.5 Test Cases

Test Case ID:TC-01

Test Case Name: Register New User

Test Priority: High

Preconditions: Email not already registered. App and API running.

Post conditions: New user is created.

User is logged in and lands in Documents (per register flow).

Table 6.1

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Open register	http://localhost:3000/register	Form with email, password, terms	Form shown	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Page load OK 2
2	Fill valid data and accept terms	New email; password rules + special char; checkbox on	No validation error on submit	Validation OK	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Matches form rules
3	Submit	Click create	Account created; auto-login; redirect to /documents	Created + redirect	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : End-to-end O

Test Case ID:TC-02

Test Case Name: Register Duplicate Email

Test Priority: Medium

Preconditions: Same email already exists.

Post conditions: Registration rejected; no duplicate user.

Table 6.2

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Submit register with existing email	Email already in DB; valid password; terms	Error e.g. "Email already registered"	Error shown	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Backend rejects duplicate

Test Case ID: TC-03**Test Case Name: Login with Valid Credentials****Test Priority:** High**Preconditions:** User account already registered in system.**Post conditions:** User is authenticated and redirected to Documents page.**Table 6.3**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Open the login page	URL: http://localhost:3000/login	Login form loads (email, password, Sign In)	Login form loads	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Page load OK 2
2	Enter valid email and password (meets app rules)	Email: registered email; Password: ≥ 8 characters and contains ≥ 1 special character (as on login form)	Fields accept input; no client-side error	Input accepted; no error	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Validation as per UI
3	Click Sign In	—	User receives token; redirect to documents area e.g. /documents	Redirect to /documents (or next param if used)	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Session works

Test Case ID:TC-04

Test Case Name: Login with Wrong Password

Test Priority: High

Preconditions: Registered user exists. App and API running.

Post conditions: User is not logged in.

Error message indicates incorrect credentials (no access to protected pages).

Table 6.4

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Open the login page	URL: http://localhost:3000/login	Form visible	Form visible	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Page load OK 2
2	Enter valid email, wrong password	Email: registered; Password: valid format but wrong	Login fails	Error from API: incorrect email/password (banner)	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Expected

Test Case ID: TC-05**Test Case Name: Upload Valid Document (PDF)****Test Priority:** High**Preconditions:** User logged in. File < 10 MB, .pdf.**Post conditions:**

File appears in All Files table.

Document can be opened for chat.

Table 6.5

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Go to Documents	http://localhost:3000/documents	Library page; upload area	Page loads	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : -
2	Upload PDF (picker or drag-drop)	Valid .pdf < 10 MB	Upload succeeds; list refreshes; row with name, type, size, date	New row visible	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Accept: .pdf in UI
3	Open document	Click Open on row	Navigate to /documents/{id}	Opens doc chat view	Chrome on Windows 10/11	PASS	[Usman 23/04/2026] : Route corr

Test Case ID: TC-06

Test Case Name: Reject Invalid File Type on Upload

Test Priority: High

Preconditions: Logged in. A non-allowed file e.g. .png or .zip.

Post conditions: Upload rejected; no new document in list.

Table 6.6

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Try upload	.png (or disallowed)	API returns invalid file type; UI shows error banner	Error shown; file not added	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] Allowed: PDF, DOCX, TXT

Test Case ID: TC-07

Test Case Name: Reject File Larger Than 10 MB

Test Priority: High

Preconditions: Logged in. A .pdf > 10 MB.

Post conditions: Upload fails; no corrupt partial entry in list (or error only).

Table 6.7

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Upload large file	file.pdf > 10 MB	Error: file too large (max 10 MB)	Error displayed	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Matches server max_upload_bytes

Test Case ID: TC-08

Test Case Name: Search Documents in Library

Test Priority: Medium

Preconditions: At least 2 files uploaded. User on Documents page.

Post conditions: Table filters by filename search query (URL or header search as implemented).

Table 6.8

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Open Documents	http://localhost:3000/documents?search=keyword (or use header search)	List filters to names containing search string	Only matching files shown (or “no match” message)	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Client-side filter on filename

Test Case ID: TC-09**Test Case Name: Single-Document Chat — Ask Question and Receive Reply****Test Priority:** High**Preconditions:** Logged in; at least one document indexed. LLM/embedding config valid so RAG can answer.**Post conditions:**

User and assistant messages appear in thread.

Session is created/continued; optional citations where applicable.

Table 6.9

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Open document chat	/documents/{id}	Chat UI + document preview area	View loads	Chrome on Windows 10/11	PASS	[Ayesha23/04/2026] : —
2	Type question; Send	Question about document content	“Thinking...” then assistant message	Assistant reply received	Chrome on Windows 10/11	PASS	[Ayesha23/04/2026] : Depends on API keys
3	(If citations shown) Click citation chip	Citation e.g. [1] with page	PDF scroll/highlight to page (if feature active)	Citation works or N/A if no page	Chrome on Windows 10/11	PASS	[Ayesha23/04/2026] : PDF only for page ju

Test Case ID: TC-10**Test Case Name: Generate Document Summary****Test Priority:** High**Preconditions:** Document with extractable text.**Post conditions:** Summary text displayed in panel after generation.**Table 6.10**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	On doc chat, trigger summary	Click summary control	Loading; then “Summary” block with text	Summary appears	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : POST /documents/{id}/summary

Test Case ID: TC-11**Test Case Name: Export Summary (PDF and DOCX)****Test Priority:** Medium**Preconditions:** Document exists; user on doc page.**Postconditions:** Browser downloads summary-{id}.pdf and summary-{id}.docx (or equivalent names).**Table 6.11**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Export summary PDF	Click export summary PDF	Download starts, PDF opens	File downloaded	Chrome on Windows 10/11	PASS	[Ayesha23/04/2026] : /export/summary/{id}?format=pdf
2	Export summary DOCX	Click export summary DOCX	Download DOCX	File downloaded	Chrome on Windows 10/11	PASS	[Usman 23/04/2026] : format=doc

Test Case ID: TC-12**Test Case Name: Multi-Document Chat — Select and Ask****Test Priority:** High**Preconditions:** ≥ 2 documents in library.**Post conditions:** Chat uses selected document set; message sends only when at least one doc selected (per multi-page logic).**Table 6.12**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Open multi chat	http://localhost:3000/documents/multi	Picker + chat area	Page loads	Chrome on Windows 10/11	PASS	[Usman 23/04/2026] : —
2	Select 2+ documents	Check two filenames	Selection chips; send enabled or prompt if empty	With selection, can send	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : document_ids in API
3	Send question	Text question	Assistant reply (multi-doc RAG)	Reply received	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Same LLM limits as TC-09

Test Case ID: TC-13**Test Case Name: Block Send in Multi-Chat When No Document Selected****Test Priority:** Medium**Preconditions:** On multi page; can clear selection.**Post conditions:** User sees error or send disabled; no request with empty document_ids.**Table 6.13**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Deselect all; try Send	Any question	Error: select at least one document / input disabled	Blocked with message	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Prevents bad API call

Test Case ID: TC-14**Test Case Name: Bookmark Chat Message and View in Bookmarks Page****Test Priority:** Medium**Preconditions:** At least one assistant (or user) message in a session.**Post conditions:**

Bookmark API succeeds.

Entry visible on Bookmarks page with session title and snippet.

Table 6.14

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	In chat, bookmark a message	Click bookmark on message	Icon filled; created via API	Success	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : POST /bookmarks
2	Open bookmarks	http://localhost:3000/documents/bookmarks	List includes saved item	Item listed	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : GET /bookmarks

Test Case ID: TC-15**Test Case Name: Remove Bookmark from Bookmarks Page****Test Priority:** Low**Preconditions:** At least one bookmark exists.**Post conditions:** Bookmark removed from list.**Table 6.15**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	On bookmarks, remove	Click delete for bookmark	Item disappears	Removed	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : DELETE by message / book

Test Case ID: TC-16**Test Case Name: Export Chat Transcript (PDF and DOCX)****Test Priority:** Medium**Preconditions:** Active chat session with messages.**Post conditions:** Files download: chat export PDF and DOCX.**Table 6.16**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	With session open, export chat PDF	Session id present; export control	chat-{session_id}.pdf download	File downloaded	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : GET /export/chat/{session_id}?format=pd
2	Export chat DOCX	—	.docx download	File downloaded	Chrome on Windows 10/11	PASS	[Ayesha23/04/2026] : format=docx

Test Case ID: TC-17**Test Case Name: Delete Document (with Confirmation)****Test Priority:** High**Preconditions:** At least one uploaded file.**Post conditions:** Document removed from list; not openable; server deletes file and index.**Table 6.17**

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	On Documents, click delete	On a row, confirm modal	On confirm, row removed	Removed after confirm	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : DELETE /documents/{id}

Test Case ID: TC-18

Test Case Name: Access Protected Page Without Login

Test Priority: High

Preconditions: No JWT / cleared local storage.

Post conditions: User cannot view /documents without auth.

Table 6.18

SN	Action	Inputs	Expected Outcome	Actual Outcome	Test Application	Test Result	Test Comments
1	Clear site data / use incognito	—	—	—	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : —
2	Open http://localhost:3000/documents	—	Redirect to login or access denied	Redirect to /login (Require Auth)	Chrome on Windows 10/11	PASS	[Ayesha 23/04/2026] : Matches RequireAuth

6.7 Error Handling and Debugging

During testing and final integration of QueryBot, several categories of errors were observed. Most issues were handled gracefully by the application through validation messages, HTTP error responses, and user-visible banners, while a smaller set required configuration correction or code-level fixes.

6.7.1 Types of Errors Encountered

1. Client-side validation errors

The registration and login screens enforce rules before requests are sent to the server, such as email format checks and password length and complexity requirements. Invalid inputs are blocked early, which reduces unnecessary network traffic and improves data quality.

2. Server-side validation errors

The backend rejects invalid authentication attempts, duplicate registration emails, unsupported upload extensions, and files that exceed the configured maximum size. These responses are returned as structured error messages that the frontend displays in alert-style banners.

3. Authentication and authorization errors

Protected pages require a valid access token. When the token is missing or invalid, the user cannot access document and chat features and is guided back to the login flow. This prevents unauthorized access to user-specific documents and chat history.

4. File processing and indexing errors

Some documents may fail text extraction if the file is corrupted, unreadable, or effectively image-only without extractable text. In such cases, the system reports a readable failure message rather than silently accepting a broken document. Vector indexing may also fail for edge cases; the system is designed to log such failures and allow recovery paths such as re-upload or administrative re-indexing where applicable.

5. Integration and external dependency errors

The intelligent question answering and summarization features depend on external language model services and embedding pipelines. Network timeouts, invalid API keys, incorrect model identifiers for the selected provider, or provider outages can cause assistant replies to fail or fall back to diagnostic messaging. These failures are treated as environment or configuration issues rather than core UI defects.

6.7.2 Resolution Approach

- Validation-related problems were resolved by aligning frontend rules with backend constraints so that users receive consistent guidance. Authentication issues were resolved by verifying token storage, login flow, and protected route guards.
- File-related failures were addressed by testing with multiple document types (PDF, DOCX, TXT) and confirming that only supported formats within the size limit are accepted. When extraction failed, testers used alternative files or corrected document content to confirm whether the issue was document-specific.
- LLM-related failures were resolved by verifying environment variables, provider base URL, model name compatibility, API key permissions, and server connectivity. After correcting configuration, the same test scenarios were repeated to confirm stable responses.

6.7.3 Debugging Tools and Practices

Debugging was performed using a combination of:

- Browser developer tools to inspect network requests, response codes, and error payloads returned by the FastAPI backend.
- Frontend runtime logs for React and fetch errors, especially during upload, chat send, and export download actions.
- Backend server logs to capture stack traces and detailed failure reasons for indexing, parsing, and LLM calls.
- Iterative retesting after each fix to ensure that resolved defects did not reappear in related workflows.
- This approach ensured that errors were not only fixed locally but also understood in terms of root cause, which supports maintainability beyond the final demonstration.

6.8 Limitations Found During Testing

Although QueryBot achieved its intended functional scope, testing also highlighted practical limitations that are common in academic prototypes and local deployments.

6.8.1 System and Functional Limitations

1. Dependence on extractable text

The quality of retrieval and answers depends on how much meaningful text can be extracted from uploaded files. Scanned PDFs or image-heavy documents may

yield limited text, which reduces answer quality even when upload succeeds.

2. Browser preview constraints

Certain formats such as Word documents may not render inline in the browser the same way native PDF previews do. Users may need to download or open converted previews externally, which is acceptable for a prototype but affects seamless “all-in-browser” experience.

3. Single-user local deployment assumptions

The system is validated on a local developer setup. While it correctly isolates documents per authenticated user, large-scale multi-tenant performance was not a primary testing target in this phase.

6.8.2 Performance and Reliability Constraints

1. Latency variability

Response time for chat and summary generation varies with document size, chunk retrieval settings, and external model latency. Under heavy prompts or large documents, users may experience longer waiting periods.

2. External service availability

Because the assistant relies on external APIs, temporary outages or rate limits can interrupt question answering even when the local application is healthy.

6.8.3 Areas for Improvement

Future iterations could improve the system by adding automated regression tests, stronger monitoring for LLM failures, richer preview handling for office formats, OCR support for scanned documents, and deployment on cloud infrastructure with a scalable database. These improvements align naturally with the recommendations presented in the conclusion and future work chapter.

CHAPTER 07

CONCLUSION AND FUTURE WORK

7.1 Introduction

This chapter concludes the QueryBot project by summarising what was delivered against the objectives defined earlier in the report, describing the main challenges faced during development and testing, and outlining realistic directions for future enhancement. QueryBot was implemented as a web application that allows registered users to upload documents, ask natural-language questions with retrieval-augmented generation, generate summaries, export outputs, and manage bookmarks, with support for both single-document and multi-document conversations.

7.2 Summary of Findings and Achievements

The main outcomes of the project are as follows.

1. Functional delivery

The system provides end-to-end workflows for user registration and login, secure access to personal document libraries, upload of common office and text formats within configured limits, conversational question answering grounded on the user's own files, document-level summarisation, and export of chat and summary content. Bookmarking of important responses and a dedicated bookmarks view improve reuse of information across sessions.

2. Technical architecture

A separation between a React-based frontend and a FastAPI backend was adopted, with token-based authentication, relational storage for users and metadata, and a vector store for semantic retrieval. This structure supports clear responsibilities: the API handles validation, persistence, and AI orchestration, while the client focuses on usability, feedback, and file interactions.

3. User experience

The interface supports typical actions such as drag-and-drop upload, document search in the library, confirmation before destructive operations, and visible loading and error states during long-running tasks such as chat replies and summary generation.

Overall, the implemented prototype demonstrates that a small-team or individual deployment can offer practical document intelligence features without requiring a full enterprise stack, while still exposing the integration challenges that appear when combining file parsing, retrieval, and external language models.

7.3 Challenges and Limitations

Several challenges appeared during design, implementation, and testing. The most significant are summarised below.

1. Citation handling and PDF alignment

One of the largest challenges was making assistant answers trustworthy when the model cites evidence. The application surfaces citations (for example, numbered references tied to retrieved chunks and page numbers where available), but keeping citations consistent with the on-screen PDF preview—especially across different parsers, zoom levels, and multi-page layouts—required careful tuning and repeated manual checks. Edge cases such as tables, footnotes, or dense two-column PDFs sometimes produced snippets that were correct in meaning but harder to map visually to a single page, which affects user confidence even when retrieval is technically sound.

2. Dependence on extractable text and document quality

Answer quality is limited by how much clean text can be extracted from each upload. Image-only or heavily scanned PDFs may yield little text, which weakens retrieval regardless of model strength. This is a limitation of the current pipeline rather than of the user interface alone.

3. Latency and external AI services

Chat and summarisation depend on external language model APIs and embedding services. Response time varies with document size, number of retrieved chunks, and provider load. Failures due to keys, quotas, or network issues must be surfaced clearly; they are not always distinguishable to end users from “wrong answers,” which complicates support and evaluation.

4. Local deployment and scale

The project was validated in a local development configuration. While adequate for final-year demonstration, this setup does not by itself prove behaviour under many concurrent users or very large corpora. Database and vector-store choices that are convenient for prototyping may need redesign for production scale.

5. Consistency between marketing UI and backend limits

The product includes a Pricing page that describes tiered plans (for example Free, Pro, Enterprise) with different document counts and file-size limits. In the current implementation, enforcement is centred on a single global maximum upload size and feature set for all users. Presenting future pricing tiers therefore requires not only UI work but also server-side quotas, billing integration, and per-plan limits so that what users see on the pricing screen matches what the API actually allows.

7.4 Future Work and Recommendations

The following extensions would strengthen QueryBot and align it with both user expectations and the direction suggested by the existing pricing concept.

1. Subscription, pricing, and plan enforcement

Future work should implement a real subscription and billing layer (for example integration with a payment provider) and enforce per-plan limits for documents per month, messages per day, and maximum file size, as outlined conceptually on the Pricing page. Free tiers could retain modest limits for onboarding, while paid tiers would unlock higher volume and priority processing. This would turn the current static pricing presentation into an operational product model.

2. Support for larger PDFs and long documents

The current prototype caps uploads at 10 MB to keep local storage and indexing predictable. A natural next step is tier-based or configurable limits (for example higher caps for Pro users and very large files for Enterprise), combined with chunked upload, background indexing, and progressive processing so that large PDFs do not block the interface or time out during extraction. For extremely large files, selective indexing (only certain chapters or page ranges) could be offered to control cost and latency.

3. Stronger citation and preview experience

Improvements could include richer citation panels, highlighting of matched regions in the PDF viewer where technically feasible, and clearer messaging when page-level mapping is uncertain. Optional OCR for scanned documents would also improve both retrieval and citation usefulness.

4. Reliability and operations

Adding automated tests (API integration tests and critical UI flows), structured logging and monitoring, and retry policies for transient LLM errors would improve maintainability. Deployment to cloud hosting with managed database and secrets management would support real users beyond the lab.

5. Security and compliance

Future versions should formalise password policies, rate limiting, audit logs for document access, and options for data retention and deletion that meet institutional or organisational policies, especially if Enterprise features such as SSO are pursued.

7.5 Conclusion

QueryBot successfully combines document management with retrieval-augmented question answering, summarisation, exports, and bookmarking in a coherent web application. The project met its core functional goals while also exposing non-trivial challenges—particularly around citations, PDF behaviour, and reliance on external AI services—that are typical of modern intelligent document systems.

With planned work on real pricing and quota enforcement, support for larger PDFs through scalable ingestion, and incremental improvements to citations, testing, and deployment, the prototype provides a solid foundation for a more complete product aimed at individual learners, professionals, and eventually organisations.

